

Fundamentos de Programación

Tema 1: Aspectos básicos de la programación

Contenidos

1.- Introducción.....	2
2.- Primeros aspectos léxicos.....	2
3.- Comentarios.....	4
4.- El almacenamiento de datos.....	5
5.- Operaciones con variables.....	16
6.- Conversiones entre variables.....	32
7.- La estructura condicional.....	37
8.- Ejercicios finales.....	50

Tema

1

ASPECTOS BÁSICOS DE LA PROGRAMACIÓN

En este tema realizaremos nuestros primeros programas. Como es lógico, no harán nada espectacular, pero servirán para conocer los primeros elementos básicos de los lenguajes de programación.

1.- Introducción

Los primeros pasos en el mundo de la programación siempre son los más difíciles, ya que para hacer programas tenemos que adaptar de nuestra forma de pensar a la manera en la que “piensa” el ordenador. Se necesita, por tanto, un periodo de aprendizaje y adaptación en el que la realización constante de programas es fundamental para conseguir resultados.

Todo el mundo está capacitado para programar ordenadores, pero hay personas que tienen más facilidad que otras. A quien le cuesta más trabajo, necesita practicar más para interiorizar los conceptos fundamentales y saber aplicarlos.

Por tanto, nuestro aprendizaje de la programación se realizará poco a poco a lo largo de varios temas, y en este nos centraremos en estudiar lo más simple: la estructura que tienen los programas, los elementos básicos que intervienen en ellos, y algunas acciones sencillas como la impresión de datos por pantalla y la realización de cálculos aritméticos. Como resultado, realizaremos programas de consola muy simples para resolver problemas matemáticos de 2º ESO.

2.- Primeros aspectos léxicos

Para estudiar las primeras características del lenguaje Java, vamos a partir del código fuente del programa “Hola mundo”, cuya finalidad es mostrar un mensaje en la pantalla.

Supongamos que tenemos un archivo llamado **ProgramaHolaMundo.java** que contiene este código fuente escrito correctamente:

```
public class ProgramaHolaMundo{  
    public static void main(String[] args){  
        System.out.println("Hola mundo");  
    }  
}
```

Sobre este fragmento de código fuente tan pequeño podemos darnos cuenta de algunas características muy importantes de la escritura de programas en lenguaje Java:

- Java es un lenguaje sensible a mayúsculas. Esto significa que por ejemplo “Main”, es una palabra diferente de “main”, o que “String” es una palabra distinta de “string”. Por ejemplo, si en el código fuente cambiamos “String” por “string”, al intentar compilar veremos que el compilador nos avisa de que hemos cometido un error de escritura:

```
ProgramaHolaMundo.java:2: error: cannot find symbol
  public static void main(string[] args){
                        ^
  symbol:   class string
  location: class ProgramaHolaMundo
1 error
```

- Dentro del bloque de llaves que comienza en la línea de la palabra **main**, se escriben todas las órdenes que le damos al ordenador. Cada una va en un renglón diferente, y termina en punto y coma

Por ejemplo, si queremos que la frase “Hola mundo” aparezca tres veces en la pantalla escribiremos lo siguiente:

```
public class ProgramaHolaMundo{
    public static void main(String[] args){
        System.out.println("Hola mundo");
        System.out.println("Hola mundo");
        System.out.println("Hola mundo");
    }
}
```

Al compilar y ejecutar el programa, veremos que el mensaje “Hola mundo” aparece tres veces consecutivas en la pantalla.

```
Hola mundo
Hola mundo
Hola mundo
```

- Como puede deducirse de lo anterior, el programa se ejecuta de arriba hacia abajo, comenzando su ejecución en la primera línea que hay tras abrir el bloque main, y finalizándola al llegar a su llave de cierre.

```
public class ProgramaHolaMundo{
    public static void main(String[] args){
        System.out.println("Hola mundo");
        System.out.println("Hola mundo");
        System.out.println("Hola mundo");
    }
}
```

Instrucciones del programa

- El nombre del programa es lo que hay escrito tras las palabras “public class”, y de hecho, en el lenguaje Java¹ el archivo debe llamarse exactamente igual que él y terminar con la extensión .java

```
public class ProgramaHolaMundo{
    public static void main(String[] args){
        System.out.println("Hola mundo");
        System.out.println("Hola mundo");
        System.out.println("Hola mundo");
    }
}
```

Nombre del programa (y del archivo)

- Resumiendo todo lo anterior, podemos decir que la estructura mínima que tendrán todos los programas Java será la siguiente:

```
public class {
    public static void main(String[] args){
        }
}
```

Nombre del programa (y del archivo)

Instrucciones del programa

Ejercicio 1 Haz un programa Java llamado Datos_Juan, Datos_Ana... según tu nombre, que muestre por pantalla tu ficha personal: dni, nombre, apellidos, fecha de nacimiento, edad, dirección, localidad, teléfono y email mostrando cada dato en una línea diferente. Compila y ejecútalo para comprobar que funciona.

3.- Comentarios

Los **comentarios** son frases en formato libre que podemos escribir en cualquier lugar del programa, y que ayudan a explicar lo que hace el código fuente. Son muy útiles cuando tenemos que retomar un programa que hicimos hace tiempo, cuyos detalles se nos han olvidado.

Poner comentarios siempre es una buena práctica, y en las empresas se hace imprescindible, ya que alguien puede comenzar un programa y un compañero tener que modificarlo más adelante. Así, los comentarios le ayudarán para comprender lo que hizo la persona que lo programó.

Cuando compilamos el programa, lo primero que hace el compilador es retirar todos los comentarios. Por tanto, podemos poner sin miedo todos los comentarios que queramos, ya que no tendrán ningún efecto sobre el producto final.

¹ Esto es una característica que solo encontraremos en el lenguaje Java. En otros lenguajes lo normal es que el nombre del archivo sea libre y no tenga que ser exactamente igual al nombre del programa.

Java incorpora los dos tipos de comentarios comunes a casi todos los lenguajes de alto nivel:

- Comentario de línea: Es un texto libre que se extiende hasta el final de su renglón. Para ponerlo escribimos dos barras //y todo el texto que va a continuación en la misma línea se considera comentario:

```
public class ProgramaHolaMundo{  
    public static void main(String[] args){  
        // La siguiente línea muestra "Hola mundo" en la pantalla  
        System.out.println("Hola mundo");  
    }  
}
```

- Comentario de bloque: Es un comentario que abarca varias líneas. Comienza con los caracteres /* y termina con los caracteres */

```
public class ProgramaHolaMundo{  
    public static void main(String[] args){  
        /* La siguiente línea muestra  
           "Hola mundo"  
           en la pantalla */  
        System.out.println("Hola mundo");  
    }  
}
```

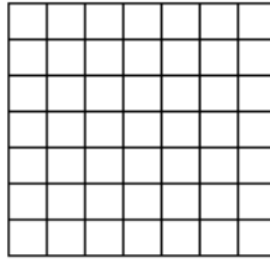
Ejercicio 2: En el ejercicio 1 coloca un comentario de bloque al principio explicando lo que hace el programa, el nombre del autor, el número de versión del programa y la fecha en que se programó. Añade comentarios de línea que consideres apropiados para explicar lo que va haciendo el programa.

4.- El almacenamiento de datos

El objetivo de todo programa es dar solución a una necesidad que encontramos en la vida real. Como es lógico, el programa necesitará trabajar con datos de muy diverso tipo (números, letras, frases, etc) que el ordenador deberá procesar. En los siguientes apartados vamos a ver cómo hacer que un ordenador trabaje con datos en un programa.

4.1.- Variables

Todos los datos que maneja el ordenador se guardan en su memoria RAM, que funciona como si estuviese formada por una serie de casillas o celdas de igual tamaño donde se guardará la información. La memoria está lista llenarse con los datos que vayamos a usar en nuestro programa.



Esquema de la estructura de la memoria

Para guardar datos en la memoria, en todos los lenguajes de programación existe el concepto de **variable**:

Una **variable** es una **zona de la memoria** que **posee un nombre** y en la que **se almacena un dato**

Es decir, una variable es una zona de la memoria (que puede ocupar varias casillas consecutivas) en la que se guarda un dato que haga falta en el programa. La cantidad de casillas que ocupa la variable dependerá del tamaño del dato que se va a guardar. Por ejemplo, almacenar la frase "hola" ocupará menos casillas que almacenar la frase "hola mundo".

Veamos ejemplos de variables que podríamos encontrar en nuestro programa:

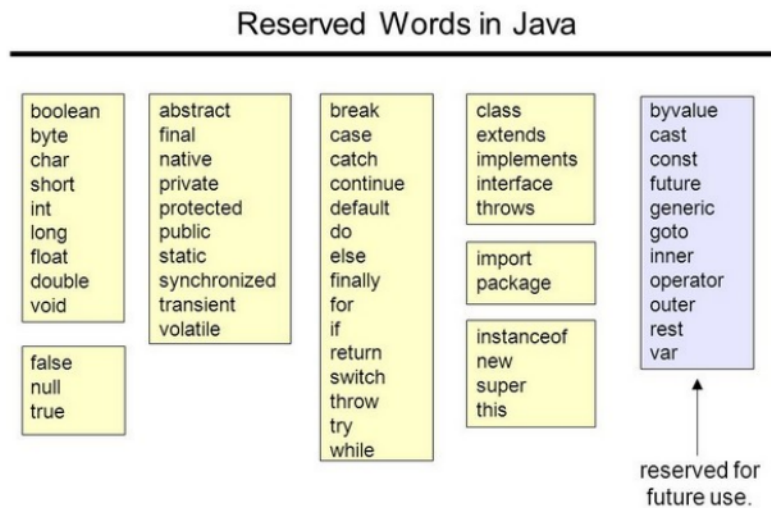
- Si en un programa necesitamos trabajar con el sueldo de una persona, podemos hacer una variable que se llame **sueldo**, y asignarle el valor de dicho sueldo, por ejemplo, **1200**
- Si esa misma persona tiene una dirección y el programa necesita trabajar con ella, podemos hacer una variable llamada **dirección** con su valor, por ejemplo, **C/ Parque Manuel S/N**
- Si en el programa también se necesita trabajar con la altura de la persona, podemos crear una variable en la memoria llamada **altura** y guardar su dato, por ejemplo **1.72**
- Si en el programa se necesita saber si la persona es un becario, podemos hacer una variable llamada **becario** y guardar en ella el valor **si**.

Las variables no se colocan de manera ordenada en la memoria, sino que es el **sistema operativo** quien decide dónde se colocarán, según el tamaño que ocupan. Por ejemplo, la disposición en la memoria RAM de las variables creadas anteriormente en los ejemplos podría ser así:

								si	
								becario	
		C/ Parque Manuel S/N							
		dirección							
								1.72	
								altura	
								1200	
								sueldo	

Con respecto a los nombres que ponemos a las variables, son libres y podemos utilizar cualquier nombre que queramos, pero tienen que cumplir las siguientes condiciones:

- El nombre tiene que empezar por una letra, un subrayado o el símbolo del dólar.
- No se pueden poner espacios en el nombre de una variable ni símbolos extraños².
- El nombre no puede superar los 64 caracteres.
- Hay una serie de palabras llamadas “palabras reservadas” que tienen significado propio y, por tanto, no se pueden usar como nombres de variables:



Como es lógico, el nombre de una variable tiene que tener relación con el contenido que almacena. Por ejemplo, si en un programa trabajamos con el nombre de un cliente, para guardarlo podemos usar una variable llamada `NombreCliente` o bien, `Nombre_Cliente`. También podríamos ponerle a esa variable el nombre `Aj23Qm3`, pero no sería apropiado porque no refleja bien lo que guarda y eso dificultaría enormemente el posterior mantenimiento del programa..

Aunque no es obligatorio, el nombre de la variable se escribirá siempre en minúscula. Cuando el nombre de la variable incluye espacios, hay varias opciones. Por ejemplo, si queremos hacer una variable que guarde el nombre completo (nombre y apellidos) de una persona, podríamos usar estos nombres:

Camel case	<code>nombreCompleto</code>
Pascal case	<code>NombreCompleto</code>
Snake case	<code>nombre_completo</code>
Screaming snake case	<code>NOMBRE_COMPLETO</code>
Kebab case	<code>nombre-completo</code>

Aunque parezca sorprendente la elección del sistema no es libre. Un buen programador utiliza la notación recomendada por el lenguaje que utiliza. Por ejemplo, en Java todos los programadores usan el sistema **camel case**

² Java admite el uso de la ñ y los acentos. Pero para que el compilador los reconozca, debemos configurar nuestro editor de texto para que guarde en el sistema de codificación **ANSI** en Windows, o **UTF-8** en Linux.

Ejercicio 3: Indica qué nombre de variable usarías para guardar los siguientes datos:

- a) El número de páginas de un libro.
- b) Si un alumno llamado Antonio es repetidor o no.
- c) La suma de los cuatro lados de un rectángulo.
- d) La letra de la puerta de una casa.
- e) La suma de los primeros 500 números primos elevados al cuadrado.

Ejercicio 4: Indica cuales de los siguientes nombres de variables son incorrectos:

- a) Hidrógeno b) mensajeTexto c) \$variable d) 1erSueldo
- f) Tamaño_de_la_lista f) Número de alumnos g) P h) .porcentaje

4.2.- Los tipos de datos

Los datos que intervienen en un programa tienen naturaleza muy diversa: números, frases, texto... y por tanto, las operaciones que se podrán hacer con ellos serán distintas según su tipo. Por ejemplo, podemos multiplicar dos números, pero no tiene sentido querer multiplicar dos textos. O sea: lo que podemos hacer con las variables dependerá del tipo de dato que almacenan.

El **tipo de dato** de una variable indica al ordenador el tipo de información que guarda

En Java hay dos grandes categorías de tipos de datos: los **tipos básicos** y los **tipos referencia**.

- Los tipos básicos son tipos de datos sencillos que se encuentran predefinidos:

Nombre	Descripción	Tamaño	Rango
byte	Byte con signo	1 byte	[-128, 127]
short	Entero corto	2 bytes	[-32768, 32767]
int	Entero	4 bytes	[-2147483648, 2147483647]
long	Entero largo	8 bytes	[-9223372036854775808, 9223372036854775807]
float	Real de simple precisión	4 bytes	[3.4e-038, 3.4e+038]
double	Real de doble precisión	8 bytes	[-1.7e-308, 1.7e+308]
char	Un carácter Unicode	2 bytes	[0, 65536] caracteres multilenguaje
boolean	Valor lógico si/no	1 byte	true, false

- Los tipos referencia son tipos más complicados que estudiaremos en profundidad más adelante. En este tema, el único tipo referencia que vamos a ver es:

Nombre	Descripción	Tamaño	Rango
String	Texto genérico	Variable	Cadena de caracteres de longitud variable

La elección del mejor tipo de dato para una variable depende del programador, y normalmente habrá varias alternativas. Veamos algunos ejemplos donde necesitamos una variable y tenemos que elegir su tipo de dato:

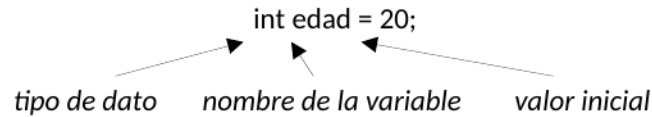
- 1) Estamos haciendo un programa en el que se debe almacenar el número de curso de un alumno. Para ello, necesitaremos una variable que pueda guardar números enteros³. Si miramos en la tabla anterior hay tres tipos posibles: byte, short o int. Como el número de curso es un valor pequeño que nunca va a sobrepasar el valor 127, lo más eficiente sería usar el tipo byte. Sin embargo en Java, por costumbre el tipo que siempre se usa para guardar un número entero, sea cual sea su tamaño, es int. Así pues, lo más eficiente sería utilizar una variable llamada númeroCurso de tipo byte, aunque en la práctica todo el mundo la pondría de tipo int.
- 2) Un programa necesita conocer la velocidad con la que se mueve un vehículo expresada en kilómetros por hora. Como este dato puede tener decimales, para guardarlo usaremos una variable de tipo real, que podría ser de tipo float o double. En Java lo más frecuente es usar el tipo double para estos datos. Por tanto, usaríamos una variable double llamada VelocidadKm_h
- 3) En un programa se trabaja con la dirección de una persona. Como este dato el dato que tenemos que guardar en memoria es una frase, lo haremos con una variable que almacene texto. En este caso la única posibilidad es usar una variable String.
- 4) Nuestro programa necesita almacenar la letra del DNI de una persona. Al tratarse de una sola letra podemos usar una variable de tipo char, pero también podríamos utilizar una variable de texto general, es decir, un String. En la práctica son válidas las dos opciones. Por tanto, utilizaremos una variable llamada letraDNI cuyo texto puede ser char o String. El uso que tengamos que hacer de esa variable en el programa nos hará decantarnos por una u otra opción.
- 5) Nuestro programa necesita trabajar con el estado civil de una persona. En principio podríamos pensar en una variable que guarde texto, que almacene los valores “soltero” o “casado”. Sin embargo, en Java tenemos el tipo boolean, que es adecuado para los casos en los que debemos guardar un “sí” (valor true) o un “no” (valor false). Por tanto, haremos una variable llamada “personaCasada” que guardará el valor true en caso de que la persona esté casada y el valor false si está soltera. Observa como el nombre de la variable nos sugiere que un true significa que la persona está casada.
- 6) Necesitamos guardar la fracción de alumnos aprobados en un curso, como por ejemplo 20/27. Si miramos la lista de tipos que hemos visto veremos que no hay ninguno que la represente exactamente. Incluso se nos podría ocurrir guardarla como si fuese un texto, pero esto sería una mala idea porque entonces no podríamos hacer operaciones matemáticas con ella. La solución óptima (por ahora) consiste en utilizar dos variables, una para almacenar el numerador y otra para el denominador. Como en este ejemplo el numerador y el denominador hacen referencia a alumnos, que son valores sin decimales, el tipo adecuado para ellas será el int.

Ejercicio 5: Escribe el tipo de dato que pondrías a las variables del ejercicio 3.

³ Recordamos que los números enteros son aquellos que no tienen cifras decimales, tanto positivos como negativos. $\mathbb{Z} = \{ \dots, -3, -2, -1, 0, 1, 2, 3, \dots \}$

4.3.- Creación de variables

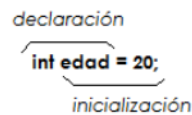
Ha llegado el momento de ver cómo se puede crear una variable en un programa Java. Para ello dentro del main se escribe una línea en la que aparece el tipo de dato, el nombre de la variable y el valor inicial que contendrá, tal y como se indica en el siguiente ejemplo:



El siguiente programa da una orden (que resaltamos en amarillo) al ordenador para que cree en su memoria una variable llamada edad con valor inicial 20:

```
public class Ejemplo1{  
    public static void main(String[] args){  
        int edad = 20;  
    }  
}
```

Si analizamos la línea que crea la variable, vemos que podemos dividirla en dos partes. La parte a la izquierda del igual se denomina **declaración**, y la que hay a la derecha **inicialización**.



En un programa podemos crear todas las variables que necesitemos, procurando no abusar de variables innecesarias, ya que consumen memoria. Una vez que una variable ha sido creada, puede utilizarse en operaciones, cálculos, etc., sin necesidad de volverla a crear de nuevo.

4.4.- Las constantes

En muchas ocasiones trabajaremos con datos que sabemos que nunca cambiarán. Por ejemplo, si necesitamos trabajar con el número de días de la semana, usaremos una variable llamada `numeroDiasSemana` que tendrá un valor de 7. Si necesitamos trabajar con el número pi usando sus primeros 4 decimales, sabemos que siempre será 3.1415.

```
public class Ejemplo2{  
    public static void main(String[] args){  
        int diasSemana = 7;  
        double pi = 3.1415;  
    }  
}
```

El problema de usar variables para estos datos es que accidentalmente pueden ser modificados y entonces contener un valor incorrecto. Para evitar este problema, Java permite “bloquear” la variable para que solo se pueda leer y no modificar. Estamos creando de esa forma, una **constante**⁴.

La creación de una constante en Java es similar a la de una variable. Se colocará la palabra **final** en su declaración y escribiremos el nombre de la constante con el sistema “screaming snake case” (mayúsculas con guiones bajos para separar palabras).

```
public class Ejemplo2{
    public static void main(String[] args){
        final int DIAS_SEMANA = 7;
        final double PI = 3.1415;
    }
}
```

Hoy día se considera una buena práctica de programación el uso de constantes, ya que evita los problemas mencionados. Tanto es así que los lenguajes actuales recomiendan guardar en constantes todos los valores que sepamos que no van a cambiar en nuestro programa, aunque no se correspondan con valores constantes reconocidos del mundo real (como pi o la cantidad de días de la semana).

Por ejemplo, si necesitamos guardar el nombre del instituto instituto “IES HLanz” y estamos totalmente seguros de que ese dato nunca jamás lo vamos a cambiar en nuestro programa, podemos guardarlo como una constante así:

```
public class Ejemplo2{
    public static void main(String[] args){
        final String NOMBRE_INSTITUTO = "IES HLanz";
    }
}
```

Como veremos más adelante, el uso de constantes también facilita la comprensión del programa, ya que sus nombres (DIAS_SEMANA, PI, etc) nos ayudan a saber lo que contienen y nos facilitan la comprensión de los cálculos matemáticos en los que puedan intervenir. Por ejemplo:

$$\frac{\text{notaProg} + \text{notaBd} + \text{notaIpe}}{3} \qquad \frac{\text{notaProg} + \text{notaBd} + \text{notaIpe}}{\text{TOTAL_NOTAS}}$$

Como podemos ver, la fórmula de la derecha se entiende mucho mejor porque el número 3 está guardado en una constante que da sentido a la fórmula.

⁴ Otros lenguajes como C o C++ permiten crear otro tipo de constantes más óptimas, que no ocupan memoria, y que son un “alias” para un valor fijo conocido de antemano. Java no permite crear ese tipo de constantes.

4.5.- Los literales

Cuando creamos una variable tenemos que darle un valor inicial, que escribimos “a mano”. La forma de escribir ese valor dependerá del tipo de dato de la variable, según estas normas:

- **int, short, byte:** Escribimos el número directamente, teniendo en cuenta que debe ajustarse al rango definido por cada tipo de dato.

```
int edad = 20;  
byte numeroHijos = 0;  
short sueldo = 900;
```

- **long:** Se escribe el número directamente como antes, pero los números muy grandes que sobrepasan el tipo int deben terminar con una L (mayúscula o minúscula).

```
long contraseña = 15126212;  
long likesFacebook = 1529621262341225L;
```

- **double:** Escribimos un número usando el punto . para separar la parte entera de la decimal. También se puede escribir en notación científica usando la letra e para indicar 10^x

```
double peso = 68.5;  
double temperatura = 0.3655e2;
```

- **float:** Es como el double, pero terminando el número en una F (mayúscula o minúscula) y teniendo la precaución de ajustarse al rango.

```
float imc = 0.2427e2F;
```

- **char:** Se puede escribir un carácter entre comillas simples, o un número entre 0 y 65536, que sería traducido internamente al carácter cuyo código Unicode sea dicho número.

```
char letraDNI = 'Z';  
char codigoMoneda = 8364;
```

- **boolean:** Es escriben las palabras true o false según lo que se vaya a guardar en la variable.

```
boolean soltero = true;  
boolean estudiante = false;
```

- **String:** Se escribe una cadena de texto entre comillas dobles. Cuando el texto ocupa muchas líneas podemos usar la triple comilla (en Java se denomina **text block**) para escribirlo.

```
String nombre = "Francisco García López";  
String ficha = ""  
    Nombre: IES Hermenegildo Lanz  
    Dirección: Calle Albondón  
    Ciudad: Granada  
    """,
```

Todos estos valores que escribimos “a mano” cuando inicializamos las variables se denominan **literales**. En estos apuntes hemos visto solo las formas más sencillas de escribirlos, pero hay muchas más, que permiten por ejemplo, escribir números en un sistema distinto al decimal, números que llevan las cifras separadas en miles o caracteres especiales (salto de línea, tabulación...)

4.6.- Mostrar variables por pantalla

Si compilamos y ejecutamos el programa del ejercicio 5, veremos que funciona pero no hace nada más. Esto es bastante lógico ya que solo crea un montón de variables y el programa termina sin hacer nada más. Si queremos mostrar el contenido de las variables por pantalla, deberemos hacerlo nosotros mediante la orden que ya conocemos: **System.out.println**

Por ejemplo, el siguiente programa crea una variable llamada edad, con valor inicial 20, y la muestra por pantalla:

```
public class Ejemplo3{  
    public static void main(String[] args){  
        int edad = 20;  
        System.out.println(edad);  
    }  
}
```

Si lo ejecutamos, veremos que la salida se limita a mostrar el valor de la variable por la pantalla. Lo normal es que queramos que aparezca un texto previo que nos informe del valor de la variable. En ese caso, podemos usar el signo + para concatenar texto escrito entre comillas dobles con una variable.

```
public class Ejemplo3{  
    public static void main(String[] args){  
        int edad = 20;  
        System.out.println("Tu edad es: "+edad);  
    }  
}
```

Más aún, podemos seguir usando el signo + para seguir concatenando texto a la variable. Incluso podemos concatenar nuevas variables a continuación si lo deseamos.

```
public class Ejemplo3{
    public static void main(String[] args){
        int edad = 20;
        System.out.println("Tu edad es: "+edad+" años");
    }
}
```

Ejercicio 6: Repite el ejercicio 1 usando variables para almacenar los datos. ¿Qué ventajas o inconvenientes observas con respecto a dicho ejercicio?

Ejercicio 7: Haz un programa que cree todas las variables del ejercicio 3, según el tipo de dato que le indicaste en el ejercicio 5 y las muestre en pantalla.

Ejercicio 8: Haz un programa que muestre el siguiente texto por pantalla, teniendo en cuenta que los datos que aparecen subrayados deben encontrarse guardados en variables.

El viernes día 26 de mayo tendrá lugar la reunión de evaluación del 1º curso de programación de aplicaciones web en el IES H.Lanz de Granada. Los profesores calificarán al alumnado y se prevé que a las 20:30 horas las notas estén publicadas en el tablón de anuncios del centro.

Ejercicio 9: Realiza una copia del archivo del ejercicio anterior. Sobre la copia, haz las modificaciones oportunas para que el programa nos informe de que la reunión de evaluación de 1º de ESO será el lunes 20 de junio y que las notas se publicarán a las 13:45.

4.7.- Pedir datos por teclado al usuario

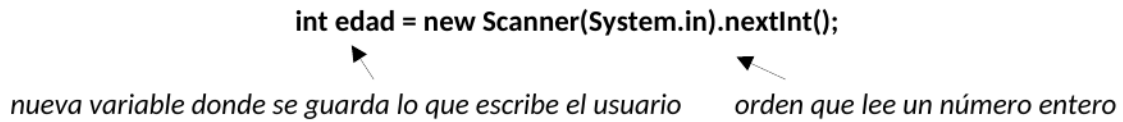
Una acción muy frecuente en todos los programas es hacer que el usuario pueda introducir datos usando su teclado. Como las aplicaciones que estamos realizando hasta ahora son de **consola**, no hay elementos gráficos para mover el ratón a un cuadro de texto y escribir en él el valor que el usuario quiere introducir.

En los programas de consola, se utiliza una orden para que el programa se espere a que el usuario escriba por teclado un dato y pulse Intro para introducirlo. En Java la orden dependerá del tipo de dato que el usuario debe introducir:

Tipo de dato que el usuario introduce	Orden que usaremos
int	new Scanner(System.in).nextInt()
long	new Scanner(System.in).nextLong()
double	new Scanner(System.in).nextDouble()
boolean	new Scanner(System.in).nextBoolean()
String	new Scanner(System.in).nextLine()

Por tanto, si queremos que el usuario escriba su edad, lo primero que haremos será poner un mensaje del tipo "Introduzca su edad" (si no lo ponemos, el usuario no sabrá que se le pide) y después usaremos la primera orden de la tabla anterior para leerlo, escribiéndola de esta forma:

`int edad = new Scanner(System.in).nextInt();`



Como podemos ver, cuando leemos por teclado un dato, necesitamos una variable (en nuestro caso **edad**) que almacene el dato que ha escrito el usuario. El programa puede terminar mostrando en pantalla un mensaje mostrando la edad del usuario.

El programa sería así:

```
public class Ejemplo4{
    public static void main(String[] args){
        System.out.println("Introduzca su edad");
        int edad = new Scanner(System.in).nextInt();
        System.out.println("Querido usuario, tienes "+edad+" años");
    }
}
```

Si intentamos compilar el programa, obtendremos un error. Eso se debe a que la orden que hemos usado para leer datos por teclado no está disponible cuando comenzamos un programa y tenemos que **importarla**.

Java viene con un montón de paquetes donde hay muchísimas órdenes que podemos importar para usarlas en nuestro programa. En nuestro caso importaremos el paquete **java.util** que es donde se encuentra la orden que hemos usado para leer datos por teclado:

```
import java.util.*;
public class Ejemplo4{
    public static void main(String[] args){
        System.out.println("Introduzca su edad");
        int edad = new Scanner(System.in).nextInt();
    }
}
```

La palabra **import** se utiliza para importar un paquete. El asterisco significa que se importa todo el contenido de ese paquete.

Una vez importado el paquete `java.util`, ya podemos compilar el programa y ponerlo en funcionamiento. El programa esperará a que el usuario introduzca su edad antes de continuar.

```
Introduzca su edad
21
Querido usuario, tienes 21 años
```

En nuestro programa, como la edad es un dato que nunca cambia (en realidad, solo se usa en el mensaje final), podríamos si quisiéramos, guardarlo en una constante. Se haría así:

```
import java.util.*;
public class Ejemplo4{
    public static void main(String[] args){
        System.out.println("Introduzca su edad");
        final int edad = new Scanner(System.in).nextInt();
        System.out.println("Querido usuario, tienes "+edad+" años");
    }
}
```

Ejercicio 10: Repite el ejercicio 8, pero el usuario introduce por teclado los datos subrayados.

Ejercicio 11: Haz un programa que pregunte al usuario su nombre, su primer apellido y su segundo apellido (cada uno será una pregunta independiente). El ordenador mostrará el nombre y apellidos del usuario en formato **apellido1 apellido2, nombre**

5.- Operaciones con variables

Una vez que ya sabemos crear variables en un programa, vamos a ver qué cosas podemos hacer con ellas. De esta forma, conseguiremos realizar acciones que procesen los datos de un programa y obtener la solución a un problema determinado.

5.1.- Asignación

Las variables se caracterizan (y se diferencian de las constantes) en que su valor puede cambiar con el tiempo. Vamos a aprender a darle una orden al ordenador para que actualice el valor de una variable y pase a valer otro distinto del valor con el que fue creada (pero manteniendo el tipo de dato de la variable, que no se puede cambiar).

- Se llama **asignación** a la operación que consiste en cambiar el valor de una variable.
- Se realiza con el signo =

Si tenemos hecha una variable llamada sueldo y queremos cambiar su valor, lo haremos así:

sueldo = 1220;

↖
nombre de la variable

↖
nuevo valor

Como vemos, para cambiar el valor de la variable simplemente se utiliza el signo igual para indicar cuál será su nuevo valor. No es necesario indicar el tipo de dato, porque el ordenador ya lo conoce de cuando se creó la variable.

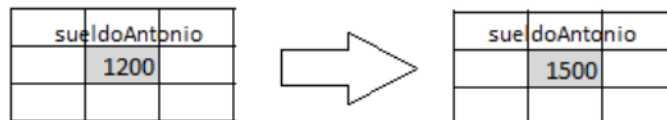
En el siguiente ejemplo creamos una variable llamada `sueldoAntonio` que guarda el valor 1200 y lo mostramos por pantalla. A continuación, le cambiamos el sueldo y lo volvemos a mostrar:

```
public class Ejemplo5{
    public static void main(String[] args){
        int sueldoAntonio = 1200;
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
        sueldoAntonio = 1500; // esto es una asignación
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
    }
}
```

Como puede verse, el programa se ejecuta de arriba abajo y la primera vez se muestra el sueldo de 1200 y la segunda vez, el sueldo de 1500.

```
Antonio gana 1200 euros
Antonio gana 1500 euros
```

En la memoria RAM lo que ha sucedido es que la casilla donde se guardaba el valor de 1200 ha sido actualizada y ahora pasa a guardar un valor de 1500.



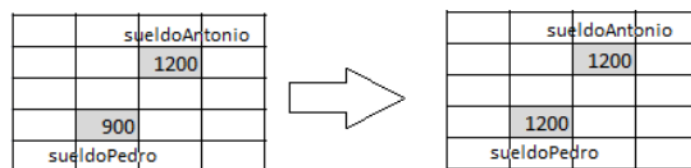
El dato que ponemos en una asignación puede ser otra variable del mismo tipo. Por ejemplo, podemos tener dos empleados y hacer una asignación para que los dos ganen lo mismo.

```
public class Ejemplo5{
    public static void main(String[] args){
        int sueldoAntonio = 1200;
        int sueldoPedro = 900;
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
        System.out.println("Pedro gana "+sueldoPedro+" euros");
        sueldoPedro = sueldoAntonio; // esto es una asignación
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
        System.out.println("Pedro gana "+sueldoAntonio+" euros");
    }
}
```

En este código creamos los sueldos de dos empleados y los mostramos por pantalla. En la línea 8 se produce una asignación que actualiza la variable sueldoPedro para guardar en ella lo que haya en la variable sueldoAntonio. Como se ve en la imagen, la variable cambia su contenido:

```
Antonio gana 1200 euros
Pedro gana 900 euros
Antonio gana 1200 euros
Pedro gana 1200 euros
```

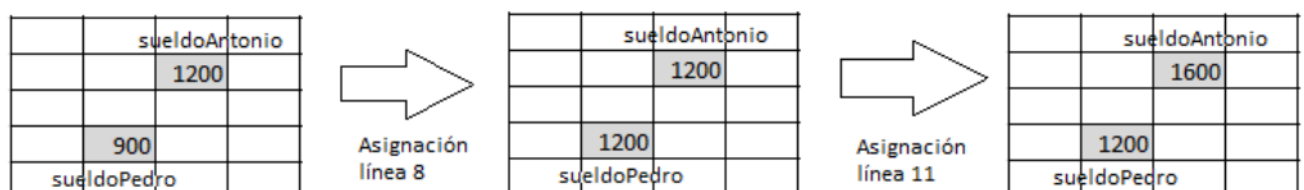
En la memoria RAM lo que sucede es que la casilla de memoria correspondiente a sueldoPedro cambia su valor y copia en ella lo que esté dentro de la casilla sueldoAntonio.



¿Y qué sucedería si a continuación del código anterior actualizásemos la variable sueldoAntonio y le asignásemos el valor 1600? ¿Cuánto valdrían los sueldos de nuestros empleados?

```
public class Ejemplo5{
    public static void main(String[] args){
        int sueldoAntonio = 1200;
        int sueldoPedro = 900;
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
        System.out.println("Pedro gana "+sueldoPedro+" euros");
        sueldoPedro = sueldoAntonio; // esto es una asignación
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
        System.out.println("Pedro gana "+sueldoAntonio+" euros");
        sueldoAntonio = 1600; // esto es una asignación
        System.out.println("Antonio gana "+sueldoAntonio+" euros");
        System.out.println("Pedro gana "+sueldoAntonio+" euros");
    }
}
```

Como en la memoria RAM cada variable ocupa su propio espacio de memoria y es independiente de las demás variables, sueldoPedro quedará como estaba en su lugar de la memoria y solo se actualizará la zona de memoria de la variable sueldoAntonio.



5.2.- Operaciones numéricas

Lo mínimo que podemos esperar de un programa es que realice las operaciones aritméticas básicas que aprendemos en el colegio. De esta forma, si tenemos dos variables numéricas⁵ cualesquiera, podemos hacer las operaciones elementales con los siguientes símbolos:

+	Suma
-	Resta
*	Multiplicación
/	• Cociente (cuando dividendo y divisor son de un tipo entero) • División con decimales (cuando dividendo o divisor no son de tipo entero)
%	Resto de la división

Una cosa **importantísima** es que cuando hacemos una operación, **obtenemos un resultado**. Como ya hemos dicho varias veces, la única forma de que el ordenador pueda trabajar con un dato es tenerlo guardado en una variable. Por tanto, **cuando hagamos una operación, deberemos guardar su resultado en una variable**, que podrá ser nueva, o ya existente. Si no hacemos esto, el ordenador no tendrá constancia del resultado de la operación.

5.2.1 – Sumas, restas y multiplicaciones

Supongamos que tenemos dos variables llamadas aula1 y aula2 que guardan la cantidad de alumnos que hay en dos aulas, y queremos obtener el número total de alumnos. Este nuevo dato es diferente al número de alumnos de cada aula, y por tanto, deberemos **crear una variable diferente** en la que se almacene el resultado de sumar el número de alumnos de cada aula. Se haría así:

int total = aula1 + aula2;

nueva variable para guardar el resultado alumnos del aula1 alumnos del aula2

El programa completo sería así:

```
public class Ejemplo6{
    public static void main(String[] args){
        int aula1 = 20;
        int aula2 = 17;
        int total = aula1 + aula2;
        System.out.println("Hay "+total+" alumnos en las dos aulas");
    }
}
```

Si lo compilamos y ejecutamos, veremos que se muestra que hay 37 alumnos, como es de esperar.

⁵ También se pueden realizar las operaciones aritméticas con variables de tipo **char**, cuyo valor es transformado automáticamente en **int** por medio de su código Unicode.

En este ejemplo hemos visto que la suma ha producido un resultado que guardamos en una nueva variable que ha sido creada para la ocasión.

Sin embargo, también podemos guardar el resultado de la suma en alguna variable que ya tengamos hecha en el programa. Usando la asignación, cambiaremos el valor de la variable para que coincida con la suma que queramos realizar.

Por ejemplo, podemos tener la variable "total" ya creada de antemano y guardar en ella el resultado de la suma, tal y como se muestra en este programa:

```
public class Ejemplo6{
    public static void main(String[] args){
        int total = 0; // aquí guardaremos el resultado de la suma
        int aula1 = 20;
        int aula2 = 17;
        total = aula1 + aula2; // asignación para guardar la suma
        System.out.println("Hay "+total+" alumnos en las dos aulas");
    }
}
```

Las operaciones de suma, resta y multiplicación funcionan de la manera habitual a la que estamos acostumbrados. Además, podemos usar paréntesis para agrupar el orden en que se realicen las operaciones⁶.

```
int operacion = (5 * (4 + 2) - 8 * (25 - 6) * 4) + 3;
System.out.println(operacion);
```

5.2.2 – La división

Las operaciones de suma, resta y multiplicación funcionan de la manera habitual a la que estamos acostumbrados. Sin embargo, un caso especial es el signo /, que en Java sirve para realizar dos operaciones matemáticas diferentes:

- **Cociente:** Es el resultado que obtenemos cuando hacemos una división sin sacar decimales
- **División real:** Es el resultado de una división cuando sacamos cifras decimales

$$\begin{array}{r} 25 \overline{) 4} \\ \underline{1} \\ 6 \end{array} \quad \leftarrow \text{cociente}$$

$$\begin{array}{r} 25 \overline{) 4} \\ \underline{0} \\ 6.25 \end{array} \quad \leftarrow \text{división real}$$

⁶ Si no se ponen paréntesis, se aplicará el orden de precedencia habitual que se usa en matemáticas

En Java se usa el símbolo / para realizar ambas operaciones. Por tanto, ¿cómo se sabe cuándo el símbolo / calcula el cociente, y cuando calcula la división real?

Todo dependerá del tipo de dato de los números que se estén dividiendo:

- Cuando los dos números que intervienen son enteros (int, short, byte, long), entonces se hace el cociente.
- Si alguno de los números que se dividen no es entero (float, double), entonces se hace la división real.

En el siguiente ejemplo calculamos la nota media de un alumno. Como las notas son de tipo double, el operador / actuará como división real y el resultado contendrá decimales.

```
public class Ejemplo7{
    public static void main(String[] args){
        double nota1 = 6.5;
        double nota2 = 4.25;
        double media = (nota1 + nota2) / 2; // dividimos double/int → hay decimales
        System.out.println("La media es "+media);
    }
}
```

En este otro ejemplo, tenemos 10 caramelos para repartir entre 4 niños y queremos saber cuántos le tocan a cada uno. En este caso, como los tipos de datos de los números que intervienen es int, el operador / actuará como cociente.

```
public class Ejemplo8{
    public static void main(String[] args){
        int caramelos = 10;
        int niños = 4;
        int cantidad = caramelos/niños; // dividimos int/int → no hay decimales
        System.out.println("Cada niño recibe "+cantidad+" caramelos");
    }
}
```

¿Qué se hace cuando queramos sacar decimales y los tipos de datos de los números sea entero? Por ejemplo, supongamos que hemos realizado tres exámenes que se han calificado como números del 0 al 10 sin decimales, y queremos obtener la media, que puede tener decimales.

El truco consiste en multiplicar por 1.0 alguno de los valores que intervienen para que se convierta en double. De esa forma, el símbolo / calculará el resultado con decimales.

```

public class Ejemplo8{
    public static void main(String[] args){
        int nota1 = 5;
        int nota2 = 7;
        int nota3 = 8;
        double media = (1.0*nota1 + nota2 + nota3) / 3; // double/int → hay decimales
        System.out.println("La media es "+media);
    }
}

```

Lo que acabamos de ver sirve siempre que queremos sacar decimales en una división entera, pero en el caso concreto del ejemplo anterior podríamos haber dividido directamente entre 3.0, porque entonces el denominador sería double y el operador / actuaría como división real.

5.2.3 – El resto de la división

Una operación muy poco utilizada en las matemáticas de los colegios, pero que aparece muchísimo en la informática (en redes, compresión de datos, criptografía...) es calcular el resto de la división de dos números.

$$\begin{array}{r}
 25 \overline{) 4} \\
 \underline{1} \\
 3
 \end{array}$$

Resto (%) $\Rightarrow$ 1 6 $\Leftarrow$ cociente (/)

Por ejemplo, si tenemos 10 caramelos y 4 niños y queremos saber los caramelos que tocan a cada niño, usaríamos el cociente. Si queremos ver cuántos caramelos han sobrado, calcularemos el resto con el símbolo %.

```

public class Ejemplo8{
    public static void main(String[] args){
        int caramelos = 10;
        int niños = 4;
        int cantidad = caramelos/niños; // cociente de dividir 10 entre 4
        int resto = caramelos % niños; // resto de dividir 10 entre 4
        System.out.println("Cada niño recibe "+cantidad+" caramelos");
        System.out.println("Sobran "+resto+" caramelos");
    }
}

```

La salida del programa produce el resultado esperado:

```

Cada niño recibe 2 caramelos
Sobran 2 caramelos

```

5.2.3 – Incrementos y decrementos

Una acción muy habitual en todos los programas consiste en incrementar o decrementar el valor de una variable en una cantidad. En Java esto se hace con estos operadores:

+=	Incremento
-=	Decremento
++	Incremento unitario
--	Decremento unitario

Por ejemplo, tenemos una variable llamada `sueldoAntonio` y queremos incrementarla en 250. Usaremos el símbolo `+=` para incrementarlo así:

`sueldoAntonio += 250;`

variable que se va a incrementar operador de incremento cantidad

```
public class Ejemplo9{
    public static void main(String[] args){
        int sueldoAntonio = 1500;
        sueldoAntonio += 250; // incrementa 250 el valor de sueldoAntonio
        System.out.println(sueldoAntonio); // muestra 1750
    }
}
```

Otra forma de realizar el incremento consiste en realizar la suma que incrementa la variable, y almacenar el resultado en la misma variable que está siendo incrementada.

```
public class Ejemplo9{
    public static void main(String[] args){
        int sueldoAntonio = 1500;
        sueldoAntonio = sueldoAntonio + 250; // suma 250 a sueldoAntonio y lo guarda en ella misma
        System.out.println(sueldoAntonio); // muestra 1750
    }
}
```

Por último, un caso especial que, como veremos durante el curso, aparece innumerables veces en todos los programas es el de incrementar en una unidad. Eso se consigue con el operador de incremento unitario. Por ejemplo, si tenemos una variable llamada `ventas` que guarda el número de ventas registradas en una tienda y queremos añadir una más, haremos esto:

`ventas++;`

variable que se va a incrementar operador de incremento unitario

```

public class Ejemplo9{
    public static void main(String[] args){
        int ventas = 21;
        ventas++; // incrementa ventas en 1
        System.out.println(ventas); // muestra 22
    }
}

```

Tras la ejecución de la línea anterior, la variable **ventas** quedará incrementada automáticamente. Los operadores `-=` y `--` funcionan de forma análoga para realizar el decremento de una cantidad y el decremento de una sola unidad.

Ejercicio 12: Haz un programa que pida al usuario por teclado dos números con decimales: `precioNormal` y `porcentajeRebaja`. El programa mostrará por pantalla el precio normal y el rebajado, tras haber aplicado el porcentaje de descuento indicado en la variable `porcentajeRebaja`. La salida del programa deberá tener el siguiente formato:

```

Precio normal del artículo: ..... euros
Porcentaje de rebaja aplicado: ..... %
Descuento aplicado: ..... euros
Precio final del artículo: ..... euros

```

Ejercicio 13: Realiza un programa que pregunte al usuario por teclado un número de "euros" y el programa mostrará por pantalla el cambio de dicha cantidad a las siguientes monedas:

Dolares	1 € = 1.42 \$
Libras	100 € = 87.13£
Yens	1 € = 113.86 ¥
Pesetas	1 € = 166.386 pts

Ejercicio 14: Crea un programa que pregunte al usuario un número entero llamado `gradosCentígrados`. El programa deberá convertir dicha cantidad a grados Fahrenheit y mostrarla por pantalla. La fórmula para realizar la conversión es:

$$F = \frac{9}{5} C + 32$$

Ejercicio 15: Un videoclub alquila la película "El padrino" a 3.5 €/día y "Odisea 2001" por 2.95 €/día. Haz un programa que pregunte por teclado el número de días que queremos alquilar las dos películas y la cantidad de dinero que paga el cliente. El programa mostrará si dicho dinero es suficiente para pagar el alquiler la cantidad de días indicada.

Ejercicio 16: Haz un programa que pregunte al usuario tres números enteros y guárdalos en variables llamadas horas, minutos y segundos. El programa mostrará el número total de segundos que hay juntando las cantidades de tiempo expresadas en las tres variables.

Ejercicio 17: Un estudiante de universidad cobra 7€ por cada hora de sus clases particulares. Haz un programa que muestre por pantalla:

- ¿Cuánto ganará en un mes si tiene dos alumnos, a los que les da 2 y 3 clases semanales respectivamente.
- ¿Cuántas horas de clase debe dar al mes si quiere ganar 900€?

Ejercicio 18: Una academia contrata al estudiante del ejercicio anterior y le ofrece un sueldo mensual de 200€ fijos más el 15€ mensuales por cada alumno. Supongamos que hay 30 alumnos apuntados en la academia, que van una hora todas las tardes de lunes a viernes y que cada uno paga 100€ al mes. Realiza un programa que calcule:

- ¿Cuánto dinero gana el estudiante que da las clases?
- ¿Cuánto dinero gana la academia al mes?

Ejercicio 19: Un camión que tiene capacidad para 3 toneladas necesita 15 viajes para transportar una cantidad de arena. Realiza un programa que pregunte por pantalla la capacidad de otro camión y nos muestre por pantalla cuántos viajes necesitará para transportar esa misma cantidad de arena.

5.3.- Comparación

Imaginemos que tenemos dos variables del mismo tipo y queremos que el ordenador nos diga si guardan o no el mismo valor. Nosotros respondemos fácilmente a esta pregunta usando nuestros ojos. ¿Son iguales las variables de cada uno de estos ejemplos?

	-8
-8	

M	
	K

true	
	false

8.25	
	10.8

Como podemos ver, mirando el contenido fácilmente podemos saber si son iguales o no cada una de estas parejas de variables.

Es importante darse cuenta que la respuesta que damos a cada apartado es **si** o **no**, por lo que cuando comparamos si dos variables guardan el mismo dato, estamos haciendo una operación cuyo resultado es un boolean.

Como ya vimos, las operaciones aritméticas producen un resultado numérico. Sin embargo, la comparación es una operación que produce un resultado boolean, porque devuelve verdadero (true) cuando las variables guardan el mismo dato, y falso (false) en caso contrario. Por tanto, cuando hagamos una comparación, deberemos guardar su resultado en una nueva variable, tal y como hicimos con las operaciones que producían resultados numéricos.

La comparación es la operación que comprueba si el valor de una variable coincide con el de otra.

- La comparación produce un resultado de tipo boolean
- Se realiza con el signo ==
- Solo se puede realizar para tipos básicos (no sirve para tipos referencia)

Vamos a verlo en un ejemplo. Tenemos dos variables llamadas sueldoAntonio y sueldoPedro que guardan los sueldos de dos personas y queremos que el ordenador calcule si los sueldos son iguales o no. Escribiremos la siguiente línea:

```
boolean sueldosIguales= sueldoAntonio == sueldoPedro;
```

nueva variable para guardar el resultado sueldo del empleado1 sueldo del empleado2

El programa completo para hacer esto sería:

```
public class Ejemplo10{  
    public static void main(String[] args){  
        int sueldoAntonio = 1200;  
        int sueldoPedro = 1500;  
        boolean sueldosIguales = sueldoAntonio == sueldoPedro; // Comparación  
        System.out.println("Sueldo de Antonio: "+sueldoAntonio);  
        System.out.println("Sueldo de Pedro: "+sueldoPedro);  
        System.out.println("¿Lo sueldos son iguales? "+sueldosIguales);  
    }  
}
```

Al compilar y ejecutar, veremos que aparecen los dos sueldos y luego el resultado de la comparación, mostrando el contenido de la variable **sueldosIguales**, que en este caso es false.

```
Sueldo de Antonio: 1200  
Sueldo de Pedro: 1500  
¿Lo sueldos son iguales? false
```

Aún no hemos visto cómo se podría hacer para cambiar el último mensaje y poner en nuestro idioma “sí” o “no”. Más adelante veremos cómo poner un mensaje personalizado en lugar de “true” o “false”.

También existe otra forma de hacer la comparación, que es la inversa de la que hemos visto. En lugar de comparar si dos variables son iguales, también podemos comparar si dos variables son diferentes. En este caso, el operador que se utiliza es !=

Por ejemplo, si lo que queremos es comprobar si los dos empleados del ejemplo anterior tienen sueldo distinto, lo haríamos así:

```
public class Ejemplo10{
    public static void main(String[] args){
        int sueldoAntonio = 1200;
        int sueldoPedro = 1500;
        boolean sueldosDistintos = sueldoAntonio != sueldoPedro; // Comparación
        System.out.println("Sueldo de Antonio: "+sueldoAntonio);
        System.out.println("Sueldo de Pedro: "+sueldoPedro);
        System.out.println("¿Lo sueldos son distintos? "+sueldosDistintos);
    }
}
```

Si ejecutamos el programa, podremos ver que ahora la respuesta es justo la contraria:

```
Sueldo de Antonio: 1200
Sueldo de Pedro: 1500
¿Lo sueldos son distintos? true
```

Para finalizar, indicamos que el operador `==` solo sirve para realizar la comparación de tipos básicos, y **no sirve para tipos referencia**. Por tanto, en Java **no podemos comparar dos Strings con el operador `==`** porque eso funcionaría unas veces si y otras no, como se verá en próximos temas.

```
public class Ejemplo11{
    public static void main(String[] args){
        String nombre1 = "Pepe";
        String nombre2 = "Manuel";
        boolean iguales = nombre1 == nombre2; // ERROR MUY GORDO!
    }
}
```

5.3.1 – Comparaciones numéricas

Cuando las variables guardan números, es muy frecuente preguntarnos si uno es mayor o menor que el otro. En este caso estamos haciendo una **comparación numérica**⁷, que producirá como resultado un boolean. Las comparaciones numéricas se realizan con los siguientes símbolos:

<	Menor que
<=	Menor o igual que
>	Mayor que
>=	Mayor o igual que

⁷ A las variables de tipo char también se les puede hacer esta comparación, porque internamente son convertidas en int por medio de su código Unicode.

La forma de realizarlas es similar a la comparación ordinaria. Como sucedía entonces, es muy importante recoger el resultado boolean que nos da la operación en una variable.

Por ejemplo, supongamos que tenemos las variables sueldoAntonio y sueldoPedro de los ejemplos anteriores, y queremos saber si Antonio gana más que Pedro. Realizaríamos una comparación numérica de esta forma:

boolean antonioGanaMas = sueldoAntonio > sueldoPedro;

↙ ↗ ↘

nueva variable para guardar el resultado sueldo de Antonio sueldo de Pedro

Aquí tenemos el programa donde realizaríamos todo el proceso:

```
public class ProgramaHolaMundo{
    public static void main(String[] args){
        int sueldoAntonio = 1200;
        int sueldoPedro = 1500;
        boolean antonioGanaMas = sueldoAntonio > sueldoPedro; // Comparación numérica
        System.out.println("Sueldo de Antonio: "+sueldoAntonio);
        System.out.println("Sueldo de Pedro: "+sueldoPedro);
        System.out.println("¿Antonio gana más que Pedro? "+ antonioGanaMas);
    }
}
```

Ejercicio 20: Realiza un programa que pregunte por teclado al usuario las tres notas de sus exámenes y nos muestre por pantalla si su media le sale aprobado.

Ejercicio 21: Realiza un programa que pregunte por teclado al usuario un número y nos diga si ese número es par.

5.4.- Operaciones con variables booleanas

El tipo de dato boolean guarda solamente dos tipos de valores, que son verdadero (true) y falso (false). Aunque parezca mentira, es posible realizar operaciones con ellos, que van a producir resultados también boolean. Estas operaciones, también se denominan **operaciones lógicas**, y fueron estudiadas en 1847 por el matemático inglés George Boole.



En la época de Boole, nadie comprendía qué utilidad práctica podía tener estudiar las operaciones que se pudieran realizar con el "verdadero" y el "falso", y fue un incompromiso. Aún así, realizó su estudio sin ser consciente de lo importante que sería un siglo después.

Las variables boolean y sus operaciones son la base de todos los razonamientos que hacen los ordenadores. Gracias a ellas, los ordenadores pueden "pensar".

5.4.1.- La operación Y

Esta operación se utiliza en programación cuando queremos detectar si dos cosas pasan a la vez.

Por ejemplo, supongamos que tenemos una discoteca y la puerta de entrada solo se puede abrir a personas mayores de edad que además tengan una tarjeta de invitación. Cuando llega una persona, el software de control de la puerta crea dos variables booleanas para registrar ambas condiciones:

```
boolean mayorEdad = true;
boolean tieneInvitacion = true;
```

A partir de estos datos, el software de la puerta debe calcular si se cumplen las dos condiciones a la vez, y para ello utiliza la operación Y, cuya definición es la siguiente:

- **La operación Y (también llamada AND), es una operación entre dos variables booleanas que produce como resultado true cuando el valor de las dos variables es true.**

&&	true	false
true	true	false
false	false	False

- **Su símbolo es &&**
- **Sirve para detectar si dos condiciones se dan a la vez**

Para hacer la operación, bastará escribir el símbolo && entre las dos variables booleanas, y como siempre, no debemos olvidarnos de recoger el valor de la operación en una variable.

```
boolean abrirPuerta = mayorEdad && tieneInvitación;
```

nueva variable para guardar el resultado indica si es mayor edad indica si tiene invitación

Como vemos, el cálculo de la operación Y entre las variables mayorEdad e invitación nos devuelve como resultado si la puerta debe abrirse. El siguiente código realizaría esta situación:

```
public class Ejemplo13{
    public static void main(String[] args){
        boolean mayorEdad = true;
        boolean tieneInvitacion = true;
        System.out.println("¿Es mayor de edad? " + mayorEdad);
        System.out.println("¿Tiene invitación? " + tieneInvitacion);
        boolean abrirPuerta = mayorEdad && tieneInvitacion; // Operación Y
        System.out.println("¿Se abre la puerta? "+abrirPuerta); // Muestra true
    }
}
```

5.4.2.- La operación O

Esta operación se utiliza cuando tenemos dos cosas y queremos detectar si alguna de ellas se cumple.

Supongamos que tenemos una tienda que ofrece un descuento a los menores de 30 años y a los parados. Imaginemos que el software que asigna los descuentos registra estas dos variables a una persona que acaba de llegar a comprar:

```
boolean menor30 = false;
```

```
boolean parado = true;
```

¿Cómo hace el ordenador para detectar si debe aplicar un descuento? Para ello, deberá utilizar la operación O, cuya definición es la siguiente:

- La operación O (también llamada OR), es una operación entre dos variables booleanas que produce como resultado true cuando alguna de ellas vale true.

	true	false
true	true	true
false	true	false

- Su símbolo es ||
- Sirve para detectar si se da (al menos) una condición entre varias

En nuestro ejemplo, para calcular la aplicación del descuento escribiríamos esto:

```
boolean aplicarDescuento = menor30 || parado;
```

nueva variable para guardar el resultado indica si es menor de 30 indica si es parado

El código completo que mostraría toda la situación sería:

```
public class Ejemplo14{
    public static void main(String[] args){
        boolean menor30 = false;
        boolean parado = true;
        System.out.println("¿Es menor de 30 años? " + menor30);
        System.out.println("¿Es parado? " + parado);
        boolean aplicarDescuento = menor30 || parado; // Operación O
        System.out.println("¿Aplicar descuento? " + aplicarDescuento); // Muestra true
    }
}
```

5.4.3.- La operación NO

Esta operación, simplemente invierte el valor de una variable booleana para obtener su opuesto, y se utiliza cuando queremos guardar lo contrario de lo que está guardado en otra variable booleana. Su símbolo es el signo !.

Por ejemplo, si tenemos una variable que guarda verdadero si una persona está casada, podemos calcular si está soltera invirtiendo su valor, así:

boolean soltero = !casado;

nueva variable para guardar el resultado operador NO indica si está casado

Aquí tenemos un programa que guarda en una variable si una persona está casada y nos muestra en pantalla si está soltera:

```
public class Ejemplo15{
    public static void main(String[] args){
        boolean casado = true;
        boolean soltero = !casado; // Operación NO
        System.out.println("¿Está casado? "+casado);
        System.out.println("¿Está soltero? "+soltero);
    }
}
```

Ejercicio 22: Realiza “a mano” los siguientes cálculos con las variables booleanas A=true, B=false, C=false y D=true

- a) $A \parallel (B \&\& C)$
- b) $(A \&\& !B) \parallel !(D \&\& A)$
- c) $A \parallel (B \&\& (D \&\& C \parallel !(A \parallel B) \&\& C))$
- d) $!(A \parallel B) \parallel (!A \&\& !B)$
- e) $(!C \&\& (!A \parallel !B)) \parallel (D \&\& !(C \&\& B))$

Ejercicio 23: Una tienda realiza descuentos a personas menores de edad y también a personas mayores de 60 años. Realiza un programa que pregunte una edad y nos muestre si tiene descuento.

Ejercicio 24: En un colegio por motivos de seguridad los niños no pueden salir solos. La puerta de entrada está programada para abrirse cuando llega un adulto, o cuando llega un niño acompañado por un familiar. Haz un programa que pregunte por teclado si la persona que llega es adulta o no, y si va a acompañada o no. Tras recoger estos datos, el programa mostrará por pantalla si la puerta deberá o no abrirse.

Ejercicio 25: Un usuario de un programa pulsa el botón de imprimir. Solo se puede realizar esta operación si la impresora está encendida, tiene tinta y el número de folios que hay en la impresora es mayor que el número de folios del trabajo que se van a imprimir. Realiza un programa que pida al usuario por teclado el porcentaje de tinta, los folios que hay en la impresora, los folios que se desean imprimir y si la impresora está encendida. El programa mostrará si se puede imprimir.

Ejercicio 26: En los ciclos de Formación Profesional, un alumno que falte más de un 20% de las horas de clase pierde el derecho a la evaluación continua.

Lenguajes de marcas	128 horas
Redes	192 horas
Hardware	96 horas

- Realiza un programa que muestre por pantalla la cantidad máxima de faltas permitidas para cada una de estas asignaturas
- Haz que el programa pregunte al usuario por el número de faltas de un alumno en redes y nos muestre si el alumno ha superado o no la cantidad de faltas indicada.

6.- Conversiones entre variables

Cuando hacemos un programa muchas veces necesitamos **guardar** el valor que hay en esa variable dentro de otra variable que tiene un tipo diferente. Por ejemplo, podría ocurrir que tuviésemos esta variable:

int sueldo = 1200

Y por el motivo que fuese (veremos situaciones en próximos temas), necesitamos guardar el valor que contiene en variables como estas:

long sueldo1

byte sueldo2

String sueldo3

¿Podemos poner directamente asignaciones como las siguientes o darán error? (hay que tener en cuenta que estamos haciendo asignaciones sobre variables que no son del mismo tipo)

```
long sueldo1 = sueldo;  
byte sueldo2 = sueldo;  
String sueldo3 = sueldo;
```

Hay lenguajes que directamente dan error cuando se intenta hacer asignaciones entre variables de distinto tipo. Otros, en cambio, hacen todo lo posible para que la asignación se pueda hacer (aunque no se mantenga el dato original). Java adopta una posición intermedia que vamos a ver en los siguientes apartados:

6.1.- Conversión implícita

La primera regla que vamos a estudiar es que el valor de una variable numérica siempre se puede guardar en otra de un tipo más grande. Esto significa que la variable **sueldo** del apartado anterior, al ser de tipo **int** se podrá guardar en variables que contengan un rango de números más grandes, como por ejemplo:

```
long sueldo1 = sueldo;  
float sueldo3 = sueldo;  
double sueldo2 = sueldo;
```

Cuando guardamos una variable de un tipo más pequeño en una de tipo más grande estamos haciendo una **conversión implícita**. Esta es la lista ordenada de los tipos de datos, de menor a mayor:

byte	short	int char	long	float	double
------	-------	-------------	------	-------	--------

¿Qué ocurre con los tipos que no aparecen en la tabla, como el boolean y el String? Pues que esos tipos no admiten conversión implícita. O sea, en Java no podemos hacer cosas como estas:

```
int sueldo = 1200;  
String sueldo2 = sueldo;  
boolean sueldo3 = sueldo;
```

Por último, mencionamos un tipo muy curioso de conversión implícita, que es cuando aparece el tipo **char**. Como hemos visto, este tipo representa un carácter. En el lenguaje Java siempre que un char aparece en una operación (suma, resta, etc) automáticamente se transforma en tipo **int** convirtiendo el carácter por su **código Unicode**. La conversión contraria (de **int** a **char**) también se produce, siempre que el número esté en el rango de los códigos Unicode.

☞ **¿Qué es el código Unicode?** Los ordenadores internamente representan las letras como si fuesen números. O sea, cada letra (o símbolo) tiene asociado un número único. Unicode es una tabla que guarda la conversión a número de los caracteres de todos los idiomas del mundo. Para las letras que usamos habitualmente, esta tabla coincide con un sistema antiguo denominado "código ASCII".

TABLA DE CARACTERES DEL CÓDIGO ASCII																										
1	0	25	!	49	1	73	I	97	a	121	y	145	u	169	q	193	+	217	j	241	+					
2	1	26	"	50	2	74	J	98	b	122	z	146	v	170	r	194	,	218	k	242	,					
3	2	27	#	51	3	75	K	99	c	123	{	147	w	171	s	195	-	219	l	243	-					
4	3	28	\$	52	4	76	L	100	d	124		148	x	172	t	196	.	220	m	244	.					
5	4	29	%	53	5	77	M	101	e	125	~	149	y	173	u	197	:	221	n	245	:					
6	5	30	&	54	6	78	N	102	f	126		150	z	174	v	198	;	222	o	246	;					
7	6	31	'	55	7	79	O	103	g	127		151	{	175	w	199	<	223	p	247	<					
8	7	32	(	56	8	80	P	104	h	128		152		176	x	200	=	224	q	248	=					
9	8	33	)	57	9	81	Q	105	i	129		153		177	y	201	>	225	r	249	>					
10	9	34	*	58	10	82	R	106	j	130		154		178	z	202	?	226	s	250	?					
11	10	35	+	59	11	83	S	107	k	131		155		179	{	203	@	227	t	251	@					
12	11	36	,	60	12	84	T	108	l	132		156		180		204	A	228	u	252	A					
13	12	37	-	61	13	85	U	109	m	133		157		181		205	B	229	v	253	B					
14	13	38	.	62	14	86	V	110	n	134		158		182		206	C	230	w	254	C					
15	14	39	/	63	15	87	W	111	o	135	¡	159		183		207	D	231	x	255	D					
16	15	40	:	64	16	88	X	112	p	136	¢	160		184		208	E	232	y							
17	16	41	;	65	17	89	Y	113	q	137	£	161		185		209	F	233	z							
18	17	42	<	66	18	90	Z	114	r	138	¤	162	¡	186		210	G	234	{							
19	18	43	=	67	19	91	[	115	s	139	¥	163	¢	187		211	H	235								
20	19	44	>	68	20	92	\	116	t	140	¦	164	£	188		212	I	236								
21	20	45	?	69	21	93	]	117	u	141	§	165	¤	189		213	J	237								
22	21	46	@	70	22	94	^	118	v	142	¨	166	¥	190	¡	214	K	238								
23	22	47	A	71	23	95	_	119	w	143	©	167	¦	191	¢	215	L	239								
24	23	48	B	72	24	96	`	120	x	144	ª	168	§	192	£	216	M	240								

Teniendo esto en cuenta, las siguientes líneas (aunque parezcan raras), serían correctas:

```
int a = 'Z'; // la variable guarda el número 90
char b = 126; // la variable guarda el caracter ~
```

6.2.- Conversión explícita (casting)

Hemos visto que podemos guardar una variable de “tipo pequeño” en otra de “tipo grande”, pero la conversión contraria nos da un error:

```
int sueldo = 1200;
byte sueldo2 = sueldo; // byte es más pequeño que int
short sueldo3 = sueldo; // short es más pequeño que int
```

Cuando esto sucede, Java nos permite hacer un **casting**, que es una operación que recorta bytes de la variable tipo grande, de forma que se obtenga un valor que quepa dentro de la variable del tipo pequeño.

Para hacer un casting, basta con poner entre paréntesis el tipo destino deseado para el dato del “tipo grande”. El resultado del casting ya si se podrá guardar en la variable del tipo pequeño.

esta parte se llama casting

int sueldo2= (int) sueldo;

variable para guardar el resultado tipo de dato destino variable que va a ser convertida

```
int sueldo = 1200;
byte sueldo2 = (byte)sueldo;
short sueldo3 = (short)sueldo;
```

Cuando se hace casting, es posible que el dato que había en la variable de tipo grande ya no sea el mismo que se guarda en la variable de tipo pequeño, ya que se han recortado bytes de su zona de la memoria. Hay veces que los datos recortados son ceros y el dato se podrá conservar, pero esto no es algo que se pueda garantizar siempre.

La operación de casting se denomina **conversión explícita**, porque no se realiza automáticamente y el programador debe realizarla expresamente, como señal de que es consciente de lo que quiere hacer.

Java admite castings entre tipos básicos numéricos y char, pero no admite castings entre ellos y el String.

Ejercicio 27: Indica cuáles de estas conversiones son posibles y cuáles no, indicando su tipo:

<pre>// Variables int a = 8; char b = 'x'; boolean c = true; double d = 2.25;</pre>		<pre>// Conversiones int conversion1 = d; float conversion2 = c; boolean conversion3 = (boolean)a; float conversion4 = d; double conversion5 = a; short conversion6 = (short)a; long conversion7 = b; int conversion8 = (int)d; boolean conversion9 = c; char conversion10 = (char)d;</pre>
---	---	--

Ejercicio 28: En una clase de 26 alumnos ha aprobado el 66% de alumnos, ha suspendido el 19.5% y el resto no se han presentado. Realiza un programa que muestre por pantalla el número de alumnos de cada tipo.

6.3.- Conversiones entre String y tipos básicos

Un String puede ser un texto cualquiera. Sin embargo, como no hay ninguna restricción al contenido de un String, puede ocurrir que en nuestros programas tengamos datos escritos como texto dentro de un String, como por ejemplo, estos:

```
String sueldoUsuario = "1200";
String alturaUsuario = "1.72";
String aprobadoUsuario = "true";
```

Los datos anteriores son String, y por tanto, no podemos hacer ninguna operación matemática ni booleana con ellos, aunque el contenido del texto sean datos numéricos o booleanos.

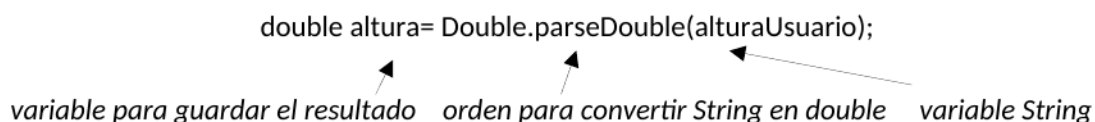
En estos casos, surge la necesidad de poder “rescatar” el número que hay dentro del String y guardarlo en una variable numérica, para así poder hacer operaciones matemáticas.

Para convertir un String (cuyo contenido textual es un número o un valor boolean) en una variable de tipo básico, utilizaremos las siguientes órdenes, según el tipo al que queramos realizar la conversión:

Integer.parseInt	Convierte un String en un int
Long.parseLong	Convierte un String en un long
Double.parseDouble	Convierte un String en un double
Boolean.parseBoolean	Convierte un String en un boolean

La forma de utilizar estas expresiones es la misma: pondremos la orden apropiada y entre paréntesis se encierra el dato String que queremos convertir. En una nueva variable recogeremos el valor de la conversión. Por ejemplo, para recuperar el número que hay dentro del texto guardado en **alturaUsuario** y guardarlo en una nueva variable llamada **altura**, escribiríamos esto:

```
double altura= Double.parseDouble(alturaUsuario);
```



De esta forma, la variable altura contendrá el valor 1.72 como número, y nuestro programa ya si podrá hacer operaciones con ella.

El resto de datos del ejemplo anterior se puede recuperar así:

```
String sueldoUsuario = "1200";
String alturaUsuario = "1.72";
String aprobadoUsuario = "true";
int sueldo = Integer.parseInt(sueldoUsuario);
double altura = Double.parseDouble(alturaUsuario);
boolean aprobado = Boolean.parseBoolean(aprobadoUsuario);
```

Es importante destacar que este tipo de conversiones de String a tipo básico solo funcionan cuando el String contiene un texto que realmente se corresponde con un dato del tipo básico al que queremos hacer la conversión. En caso de que no sea así, el programa compilará correctamente, pero al ponerlo en marcha se producirá un error⁸ indicando que no es posible realizar la conversión.

```
String sueldoUsuario = "PEPE";
int sueldo = Integer.parseInt(sueldoUsuario); // el String no guarda un número
```

En las aplicaciones profesionales lo habitual es que el usuario introduzca sus datos como un **String** (por ejemplo, en cuadros de texto de páginas web o aplicaciones de móvil) y después convertirlo al tipo adecuado. El motivo es que posible comprobar, como veremos en temas posteriores, si los String se ajustan a un formato adecuado y de esa forma, poder mostrar un mensaje de error antes de continuar haciendo cosas. Esto se denomina **validación**.

Por último, indicamos que también podemos pasar un tipo básico a String. Esto consistiría en que tenemos un valor básico (numérico, char, boolean) y queremos un String que almacene dicho valor, pero como texto. Por ejemplo:

57.25 → "57.25"

Hay varias formas de conseguir hacer eso, y la más sencilla es simplemente, pegarle a un String vacío "" el valor básico, tal y como hacemos cuando imprimimos variables en pantalla.

```
double precio = 57.25;
String precio2 = ""+precio; // guarda "57.25"
```

Ejercicio 29: Realiza un programa que pida por teclado al usuario un **String** que guarde su edad. El programa mostrará en pantalla si es mayor de edad.

⁸ Los errores que aparecen cuando el programa está funcionando se denominan **errores en tiempo de ejecución**, o en inglés, **Runtime Errors**

7.- La estructura condicional

Hasta ahora todos los programas y ejercicios que hemos hecho son **secuenciales**, esto es, se ejecutan siempre desde el principio y avanzan línea a línea hasta que se llega al final del programa.

```
public class Ejemplo16{
    public static void main(String[] args){           // inicio del programa
        System.out.println("Escribe tu nombre:");    // 1ª instrucción
        String nombre = new Scanner(System.in).nextLine(); // 2ª instrucción
        System.out.println("Hola "+nombre);          // 3ª instrucción
        System.out.println("Adios "+nombre);         // 4ª instrucción
    }                                                  // fin del programa
}
```

Sin embargo, habrá muchas ocasiones donde nos va a interesar que el programa **tome una decisión**, y haga unas acciones si sucede algo, y otras alternativas en caso contrario.

Por ejemplo, supongamos que queremos hacer un programa que pregunte al usuario dos números enteros y nos muestre el resultado de su división. Podríamos hacerlo así:

```
import java.util.*;
public class ProgramaHolaMundo{ // 17
    public static void main(String[] args){
        System.out.println("Introduce el dividendo");
        int dividendo = new Scanner(System.in).nextInt();
        System.out.println("Introduce el divisor");
        int divisor = new Scanner(System.in).nextInt();
        // calculo la división
        int division = dividendo/divisor;
        System.out.println("La división es "+division);
    }
}
```

Si ejecutamos el programa y ponemos dividendo=10 y divisor=5, todo irá bien:

```
Introduce el dividendo
10
Introduce el divisor
5
La división es 2
```

Pero cuando íbamos al instituto nos enseñaron que nunca se puede dividir un número entre cero. ¿Qué pasa si por ejemplo ponemos dividendo=10 y divisor=0? Aquí tenemos el resultado:

```
Introduce el dividendo
10
Introduce el divisor
0
Exception in thread "main" java.lang.ArithmeticException: / by zero
at ProgramaHolaMundo.main(ProgramaHolaMundo.java:9)
```

El programa ha fallado porque cuando se divide entre 0 queremos hacer una operación matemática no permitida y de ahí, el error. Como vemos, este programa no funciona correctamente en todos los casos, y da muy mala imagen a los usuarios. Este programa necesita que podamos examinar el valor de la variable “divisor” y tomar una decisión, según sea ese valor:

- Si divisor es distinto de 0, calculamos la división y mostramos el resultado en pantalla
- Si divisor es 0, mostramos un mensaje amigable de que no es posible calcular esa división

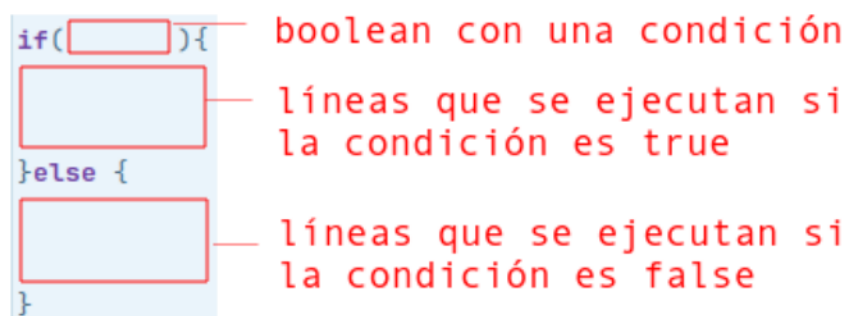
Las palabras de Java que nos van a permitir tomar una decisión en nuestro programa son **if** y **else**

7.1.- El bloque if-else

El bloque **if-else** sirve para alterar el ritmo de ejecución secuencial de un programa. Con él, el ordenador puede **tomar una decisión** y ejecutar unas líneas u otras alternativas, según el valor que tome **algo de tipo booleano**.

Básicamente consiste en que al llegar a un punto del programa, si un booleano vale "true", el ordenador ejecutará un bloque de líneas de código, y si vale "false", ejecutará otras alternativas, creándose así una especie de bifurcación dentro de la ejecución del programa.

La utilización de la palabra **if** es muy sencilla. Bastará con escribir a su lado entre paréntesis una condición cuyo tipo de dato sea booleano⁹ y a continuación, se encierran entre llaves las líneas que se ejecutarán si dicho boolean vale true. Opcionalmente se puede escribir la palabra **else** y un bloque de llaves con las líneas que se ejecutarán si el boolean vale false.

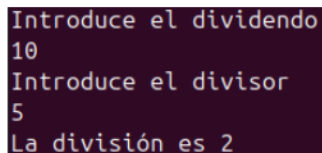


⁹ O sea, sirve cualquier cosa que podamos guardar dentro de una variable boolean. Por ejemplo, una comparación, una operación Y, etc...

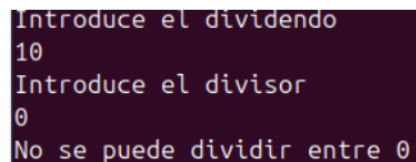
Gracias al bloque if-else es posible arreglar el programa que hicimos al principio del apartado, así

```
import java.util.*;
public class Ejemplo17{
    public static void main(String[] args){
        System.out.println("Introduce el dividendo");
        int dividendo = new Scanner(System.in).nextInt();
        System.out.println("Introduce el divisor");
        int divisor = new Scanner(System.in).nextInt();
        // tomamos una decisión en función de que el divisor sea 0
        if(divisor==0){
            System.out.println("No se puede dividir entre 0");
        }else {
            // calculo la división
            int division = dividendo / divisor;
            System.out.println("La división es " + division);
        }
    }
}
```

Si ahora el usuario introduce los valores que antes fallaban, dividendo=10 y divisor=0, el programa detectará esa situación y mostrará el mensaje puesto por el programador. Si los datos introducidos son válidos, la división se calculará correctamente.



```
Introduce el dividendo
10
Introduce el divisor
5
La división es 2
```



```
Introduce el dividendo
10
Introduce el divisor
0
No se puede dividir entre 0
```

Ejercicio 30: Realiza un programa que pregunte al usuario dos números enteros y nos diga si el primero es mayor o igual que el segundo

Ejercicio 31: Realiza un programa que pregunte un número comprendido entre 10 y 56 por teclado al usuario. Si el número está en dicho rango, el programa mostrará un mensaje de que es correcto, y en caso contrario, se indicará que no lo es.

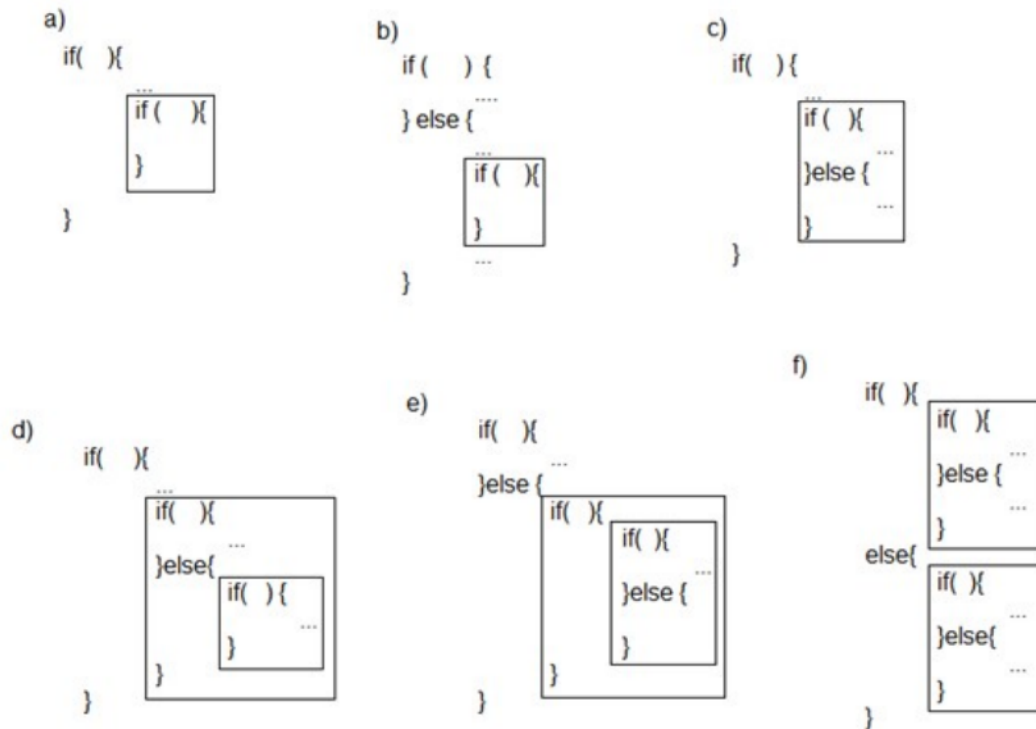
Ejercicio 32: Haz un programa que cree una constante llamada NUMERO con valor 28590. A continuación, el programa pedirá un número por teclado y mostrará en pantalla si el número introducido por el usuario es un factor del 28590 o no.

Ejercicio 33: Realiza un programa en el que haya una variable que almacena un carácter. El programa deberá mostrar por pantalla si ese carácter corresponde a una letra. Sugerencia: Convierte el carácter a número y usa la tabla ASCII

7.2.- La estructura condicional anidada¹⁰

Muchas veces al programar utilizaremos el bloque **if-else** para hacer que el ordenador tome una decisión en un punto y nos damos cuenta de que dentro del if (o del else) se necesita tomar otra decisión, y sería necesario abrir un segundo if dentro del primero. ¿Se puede hacer eso?

Pues si: es posible crear (**anidar**) nuevos bloques if-else dentro de un if o del else, sin ninguna restricción. Eso significa que podemos crear estructuras como estas:



La naturaleza del programa que estamos haciendo es la que nos dice, de forma natural, si es necesario o no anidar un if-else dentro de otro. Solo hay que comprender muy bien la situación para tomar las decisiones oportunas y anidar todos los if-else que necesitemos.

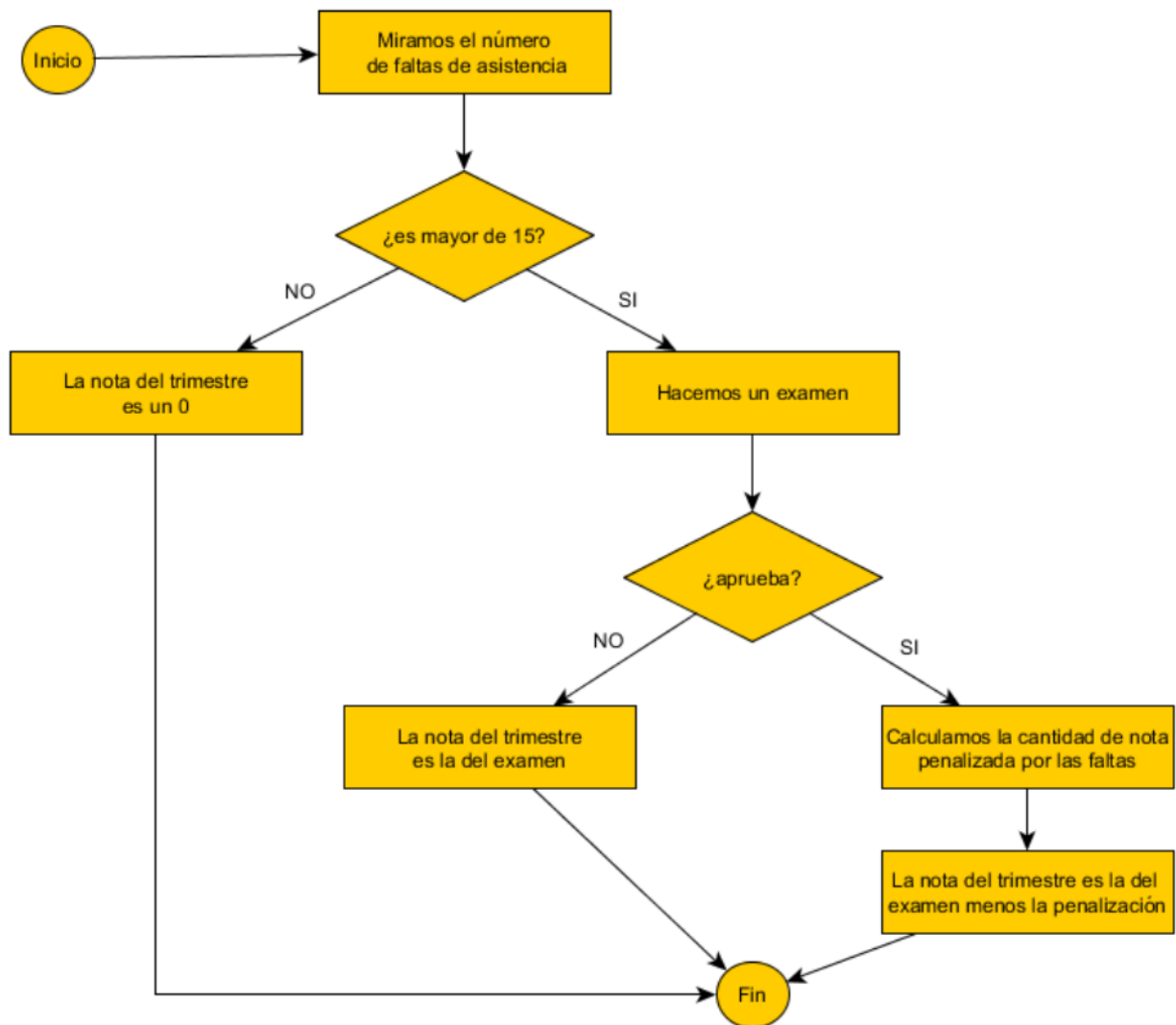
Ejemplo: Un profesor evalúa a los alumnos de la siguiente forma:

- Si un alumno tiene 15 o más faltas de asistencia no le deja hacer el examen y su nota del trimestre es un 0.
- Si un alumno tiene menos de 15 faltas, se le hace un examen y entonces:
 - Si el alumno suspende, su nota es la del examen
 - Si el alumno aprueba, su nota es la del examen menos un punto por cada 3 faltas.

¹⁰ Siempre que en programación aparece la palabra **anidado**, significa una estructura que se coloca dentro de otra.

Necesitamos un programa que ayude al profesor a obtener la nota del alumno. El programa comenzará preguntando el número de faltas y según lo que vaya pasando, irá preguntando otras cosas hasta llegar a calcular la nota final.

Solución: Un programa como este tiene ya cierto nivel de complejidad. Podemos hacer un esquema llamado **ordinograma**¹¹ que refleje la situación:



Cuando tenemos que hacer este tipo de programas es muy importante **ir poco a poco**. Si lo programamos todo del tirón y luego comprobamos si lo hemos hecho bien, lo más normal es que haya un montón de fallos y nos costaría mucho trabajo encontrarlos porque el código que hemos escrito sería muy largo y los fallos podrían estar en cualquier sitio.

¹¹ Los ordinogramas son gráficos que describen cómo es un programa de ordenador. Cada rectángulo es una parte del programa que hace algo. Los rombos representan las decisiones que hay que tomar. Las empresas usan los ordinogramas cuando tienen que describir procesos difíciles de comprender de cabeza.

Por tanto, lo ideal en estos programas más complicados es dar pequeños pasos y programar solo una pequeña parte del ejercicio. Después se prueba que lo que hemos hecho funciona. Así, si nos equivocamos, tenemos un código más pequeño que mirar y corregir. Cuando todo va bien, cogemos y repetimos el proceso programando un pequeño paso más.

Siguiendo esta norma, vamos a comenzar programando la primera parte del ejercicio: *“Si un alumno tiene 15 o más faltas de asistencia no le deja hacer el examen y su nota del trimestre es un 0”*

Para hacer esta parte escribimos código que pregunte al usuario el número de faltas de asistencia. Si el número de faltas es menor de 15, hacemos que termine el programa. En caso de que sea mayor de 15, ponemos un mensaje provisional que nos diga que eso lo haremos más adelante.

El código fuente sería este:

```
import java.util.*;
public class Ejemplo18{
    public static void main(String[] args){
        System.out.println("Escribe el número de faltas del alumno");
        int faltas = new Scanner(System.in).nextInt();
        if(faltas > 15){
            System.out.println("Tu nota es: suspenso");
        }else{
            System.out.println("TODO - Hay más de 15 faltas");
        }
    }
}
```

Como vemos, solo hemos hecho la primera parte del ejercicio. Si lo probamos, veremos que el programa funciona correctamente cuando metemos menos de 15 faltas, y si ponemos más de 15 faltas, entonces nos sale el mensaje “TODO”, que significa en inglés “to do = por hacer”.

Hecho esto ya estamos en condiciones de hacer la siguiente parte: *“Si un alumno tiene menos de 15 faltas, se le hace un examen y entonces, si el alumno suspende, su nota es la del examen”*.

Esta parte ocurre cuando el alumno tiene menos de 15 faltas. Eso se corresponde con el else donde hemos puesto el “TODO”. Así que eliminamos la línea “TODO” y nos ponemos a programar allí esa parte. El enunciado nos dice que se hace un examen y si suspende, la nota es la del examen. Por tanto, nuestro programa preguntará la nota del examen y en caso de que sea menor de 5, mostrará esa nota por pantalla. En cambio, si la nota es mayor de 5, estamos en un caso que se hará más adelante y pondremos un mensaje provisional indicando que lo haremos después.

El siguiente listado muestra el código fuente de nuestro programa provisional. Observa que la situación de nuestro programa nos lleva a programar dentro del else un nuevo bloque if-else

```

System.out.println("Escribe el número de faltas del alumno");
int faltas = new Scanner(System.in).nextInt();
if(faltas > 15){
    System.out.println("Tu nota es: suspenso");
}else{
    double nota = new Scanner(System.in).nextDouble();
    if(nota < 5){
        System.out.println("Tu nota es: "+nota);
    }else{
        System.out.println("TODO - Examen aprobado");
    }
}
}

```

Si lo probamos veremos que el código funciona correctamente: si un alumno tiene menos de 15 faltas y suspende el examen, su nota es la del examen, tal y como nos pide el enunciado. Además, el caso que habíamos hecho antes (suspender si hay más de 15 faltas) sigue funcionando. Siempre que ampliamos un programa con nuevas funciones conviene probar que las funciones anteriores siguen funcionando correctamente.

Por último, vamos a programar la parte final: *"Si el alumno aprueba, su nota es la del examen menos un punto por cada 3 faltas"*.

Esta parte se corresponde con el else que hay en la línea 11 del programa, que es el caso en que el alumno tiene menos de 15 faltas y ha aprobado el examen. En este caso la frase que tenemos que programar nos dice que hay una penalización que calcularemos restando un punto por cada 3 faltas. Esto nos dice que la penalización es el número de faltas (que está en la variable "faltas") dividido entre 3.0 (ponemos 3.0 para que dividamos entre double y la división nos de decimales). La nota final será decrementar la variable nota la penalización.

```

System.out.println("Escribe el número de faltas del alumno");
int faltas = new Scanner(System.in).nextInt();
if(faltas > 15){
    System.out.println("Tu nota es: suspenso");
}else{
    double nota = new Scanner(System.in).nextDouble();
    if(nota < 5){
        System.out.println("Tu nota es: "+nota);
    }else{
        double penalizacion = faltas/3.0;
        nota -= penalizacion;
        System.out.println("Tu nota es "+nota);
    }
}
}

```

Ejercicio 34: Realiza un programa que pregunte al usuario tres números enteros y el programa nos muestre el mayor de ellos. Resuelve el ejercicio de dos formas:

- a) Con un solo bloque if-else
- b) Con un bloque if-else anidado

Ejercicio 35: Programa este sencillo juego:

Te han secuestrado y tienes que escapar. Te encuentras en el sótano. Puedes intentar abrir la puerta o mirar alrededor. Si abres la puerta, los secuestradores te descubren y te matan. Si miras alrededor, verás una caja de herramientas y un bidón de gasolina. Puedes coger la caja de herramientas o el bidón de gasolina. Si coges la caja de herramientas puedes intentar desmontar la ventana o coger un martillo. Si intentas desmontar la ventana, lo consigues y escapas de la casa. Si cogiste el martillo, puedes intentar luchar contra los secuestradores o golpear las paredes. Si luchas contra los secuestradores, te terminan matando. Si golpeas las paredes, verás que se abre un agujero en la pared. Puedes entrar en él o intentar golpear la ventana. Si golpeas la ventana, el ruido alerta a los secuestradores, que entran y te matan. Si entras en el agujero llegas a una habitación con un cuadro eléctrico y conductos de ventilación. Puedes apagar los plomos del cuadro eléctrico o intentar salir por los conductos de ventilación. Si sales por los conductos de ventilación, ves una rata. Puedes intentar matarla o ir por otro sitio. Si la intentas matar, te muerde y mueres. Si intentas ir por otro sitio, te quedas encajado en los conductos y mueres también. Si decidiste apagar los plomos del cuadro eléctrico se va la luz y se oyen las voces de los secuestradores, que se dirigen hacia donde estás. Puedes ir golpeándolos con el martillo conforme van entrando o aprovechar la oscuridad para que cuando se abra la puerta y ellos entren, salir corriendo de la habitación. Si decides golpearlos, te cargas al primero que entra, pero los otros se dan cuenta y te matan. Si decides lo contrario, los secuestradores entran y en ese momento, sales corriendo y consigues llegar a la puerta principal, pero está cerrada con llave. Puedes mirar si la llave está en un cajón que hay al lado, o intentar golpear la puerta. Si intentas golpear la puerta, esta no se abre, los secuestradores llegan y te matan. Si abres el cajón, verás una llave. Abres la puerta y escapas. Si al principio decidiste coger el bidón de gasolina, puedes rociar con él la puerta, o puedes hacer ruido con él. Si haces ruido con el bidón, los secuestradores bajan al sótano y te matan. Si rocías la puerta con el bidón, puedes buscar si hay una cerilla o puedes esperar que se derrita la puerta. Si decides esperar que se derrite la puerta, te darás cuenta de que no pasa nada y el olor atrae a los secuestradores, que te matan. Si buscas una cerilla verás que hay una caja. Puedes prender fuego a la puerta, o puedes avisar a los secuestradores para amenazar con provocar un incendio si no te sueltan. Si prendes fuego a la puerta, se forma un incendio, toda la casa se quema y mueres. Si avisas a los secuestradores, estos te dicen que no quemes la puerta. Puedes no hacerles caso y quemar la puerta, o decirles que te suelten. Si quemas la puerta, se forma un incendio, toda la casa se quema y mueres. Si les dices que te suelten, te dicen que sí. La puerta se abre y aparecen los secuestradores armados con una metralleta. Puedes quemar la puerta, lanzarles el bidón de gasolina, abalanzarte sobre ellos o lanzarles la cerilla encendida. Hagas lo que hagas, los secuestradores te disparan y te matan.

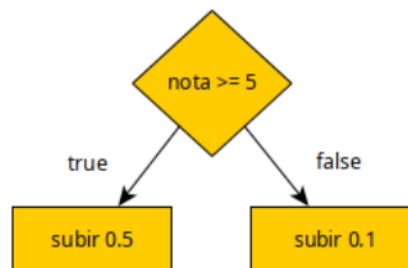
7.3.- La estructura condicional múltiple

El bloque if que hemos estudiado nos permite ejecutar una acción si se cumple una condición, y ejecutar una acción alternativa cuando no se cumple la condición.

Por ejemplo, supongamos que la nota de un alumno se calcula haciendo la media de dos notas parciales más 0.5 puntos si ha aprobado un trabajo y 0.1 si no lo ha aprobado. Lo haríamos así:

```
import java.util.*;
public class Ejemplo20{
    public static void main(String[] args){
        System.out.println("Introduzca la nota del primer examen");
        int nota1 = new Scanner(System.in).nextInt();
        System.out.println("Introduzca la nota del segundo examen");
        int nota2 = new Scanner(System.in).nextInt();
        System.out.println("Introduzca la nota del trabajo");
        int notaTrabajo = new Scanner(System.in).nextInt();
        double nota = (nota1+nota2)/2.0;
        double subidaTrabajo = 0;
        if(notaTrabajo >= 5){
            subidaTrabajo = 0.5;
        }else{
            subidaTrabajo = 0.1;
        }
        nota += subidaTrabajo;
        System.out.println("La nota del alumno es: "+nota);
    }
}
```

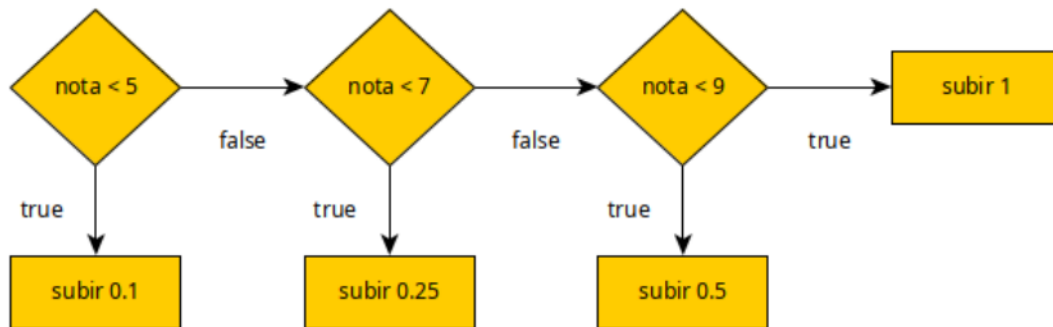
En este ejemplo vemos cómo el bloque if-else controla cuánta nota se le sube al alumno.



Sin embargo, hay situaciones en las que nos encontramos **condiciones excluyentes** (hay varias condiciones, pero están relacionadas de forma que solo puede darse una de ellas). Por ejemplo, el profesor puede subir la nota según la nota que el alumno haya sacado en el trabajo, así:

Trabajo suspenso	0.1
Trabajo aprobado con nota menor de 7	0.25
Trabajo aprobado con nota entre 7 y 9	0.5
Trabajo con nota superior a 9	1

Si nos fijamos, el cálculo de la nota que sube el trabajo consiste en tomar una cadena de decisiones analizando tres condiciones relacionadas con la nota, de la que solo una podrá ser verdadera. Una posible forma de plantear dicha cadena (hay más opciones) es esta:



Aunque podemos programar esta cadena usando if anidados¹², la forma óptima para hacerlo es utilizar un bloque **if-else if - else**. Dicho bloque funciona como si fuese un if que va evaluando condiciones una detrás de otra, y cuando encuentra una condición que se cumple, ejecuta las acciones de su if. En caso de que no se cumpla la condición, pasa a comprobar la siguiente. Al final del bloque hay un else para ejecutar acciones en caso de que ninguna condición se haya cumplido.

En el ejemplo anterior, la nota que sube el trabajo ahora se obtendría así:

```

double subidaTrabajo = 0;
if (notaTrabajo < 5) {           // analizamos si la nota del trabajo es menor de 5
    subidaTrabajo = 0.1;         // en ese caso, subimos 0.1 por el trabajo
} else if (notaTrabajo < 7) {    // si la nota del trabajo no es menor de 5, miramos si es menor de 7
    subidaTrabajo = 0.25;        // en ese caso, subimos 0.25 por el trabajo
} else if (notaTrabajo < 9) {    // si la nota del trabajo no es menor de 7, miramos si es menor de 9
    subidaTrabajo = 0.5;         // en ese caso, subimos 0.5 por el trabajo
} else {                        // si llegamos aquí, es porque la nota es mayor de 9
    subidaTrabajo = 1;           // subimos 1 punto por el trabajo
}
  
```

En el momento en que se cumple una condición, se ejecutan las acciones correspondientes al if de esa condición y el programa continúa su ejecución fuera del bloque if-else if-else. Por tanto, la comprobación de todas las condiciones excluyentes se realiza como si fuese un único bloque de condiciones en las que solo una puede ser cierta.

¹² Si usamos if anidados para programar la cadena de decisiones excluyentes, obtendremos una estructura muy compleja. Siempre es más legible leer el código sin anidamientos, todo de arriba hacia abajo, que tener estructuras anidadas que desplazan el código a la derecha.

Ejercicio 36: Realiza un programa que pregunte al usuario su nota numérica, y el ordenador muestre por pantalla la calificación obtenida según esta tabla:

Nota	Menor de 5	Entre 5 y 6	Entre 6 y 7	Entre 7 y 9	Entre 9 y 10	10
Calificación	Suspenso	Suficiente	Bien	Notable	Sobresaliente	Matrícula

Ejercicio 37: Haz un programa que nos diga el tipo de triángulo que forman tres longitudes A, B y C:

- Rectángulo: $A^2 = B^2 + C^2$
- Acutángulo: $A^2 < B^2 + C^2$
- Obtusángulo: $A^2 > B^2 + C^2$



Ejercicio 38: Realiza un programa que pregunte al usuario su año de nacimiento, si es hombre o mujer, su altura (en metros), peso (en kg) y muestre por pantalla su frecuencia cardiaca máxima, su índice de masa corporal y su estado según la Organización Mundial de la Salud.

- Frecuencia cardiaca máxima: Si es hombre se calcula como $220 - \text{edad}$, y si es mujer se calcula como $226 - \text{edad}$.
- Índice de masa corporal (IMC): Se calcula dividiendo el peso entre la altura elevada al cuadrado y se muestra con dos cifras decimales.
- El estado según la OMS se mira en esta tabla:

IMC	Estado
Menor de 16.00	Infrapeso: delgadez severa
Entre 16.00 y 16.99	Infrapeso: delgadez moderada
Entre 17.00 y 18.49	Infrapeso: delgadez aceptable
Entre 18.50 y 24.99	Peso normal
Entre 25.00 y 29.99	Sobrepeso
Entre 30.00 y 34.99	Sobrepeso: tipo I
Entre 35.00 y 40.00	Sobrepeso: tipo II
Mayor de 40.00	Sobrepeso: tipo III

Ejercicio 39: Realiza un programa para que un cliente pague en una tienda, de esta forma:

- El programa comienza preguntando el precio del artículo.
- A continuación, se preguntará si el cliente va a pagar en efectivo, con tarjeta o con Bizum.
- Si el cliente paga con tarjeta, se preguntará su pin y entonces se mostrará un mensaje con la cantidad cobrada.
- Si el cliente paga con Bizum, se preguntará su número de teléfono y entonces se mostrará un mensaje con la cantidad cobrada
- Si el cliente paga en efectivo, se pregunta la cantidad de dinero que da el cliente. (*esta cantidad puede ser mayor, menor o igual al precio del artículo*) y el programa indicará si el cliente ha pagado el precio exacto, le falta dinero o si es necesario darle el cambio. En los dos últimos casos se indicará la cantidad de dinero necesaria.

7.4.- La asignación condicional

La **asignación condicional** es una estructura que hace unos años estaba “maldita” y no se recomendaba su uso porque “hacía más difíciles de leer los programas”. Sin embargo, actualmente se ha visto su utilidad para hacer código menos propenso a fallos¹³, por lo que se ha puesto de moda otra vez. De hecho, los lenguajes de programación más recientes, incorporan estructuras parecidas a ella debido a que se ha comprobado que reducen la aparición de errores.

La asignación condicional surge para simplificar los casos en los que necesitamos un if-else para dar el valor inicial a una variable. Por ejemplo: tenemos un cine que el día del espectador hace un descuento a todos los asistentes según la edad:

Menores de edad	6€
Mayores de edad	4€

Si hacemos un programa que pregunte al usuario su edad y nos muestre el descuento vemos que necesitamos un if-else para poder dar el valor inicial a la variable **descuento**

```
import java.util.*;
public class Ejemplo19{
    public static void main(String[] args){
        System.out.println("Introduzca la edad");
        int edad = new Scanner(System.in).nextInt();
        int descuento = 0;
        if(edad >= 18){
            descuento = 4;
        }else{
            descuento = 6;
        }
    }
}
```

La asignación condicional es un operador que nos permite dar el valor a la variable **descuento**, en una sola línea, sin necesidad de utilizar el bloque if-else.

La asignación condicional en nuestro ejemplo se escribiría así:

int descuento = edad >= 18 ? 4 : 6

variable a inicializar condición valor en caso true valor en caso false

¹³ En próximos temas veremos situaciones donde si no vamos con cuidado, utilizar if nos podrá dar error, pero si usamos la asignación condicional, no.

```
import java.util.*;
public class Ejemplo19{
    public static void main(String[] args){
        System.out.println("Introduzca la edad");
        int edad = new Scanner(System.in).nextInt();
        int descuento = edad < 18 ? 6: 4;
    }
}
```

Lo que hacemos es preguntarnos si **edad<18** y si la respuesta es verdadera, asignamos el valor 6 a la variable **descuento**, y en caso contrario, el valor 4. El signo : se usa para separar el valor que se guardará en la variable si la condición es verdadera del valor que se guardará si la condición es falsa.

También podemos usarla cuando tenemos tres o más condiciones. Por ejemplo, ahora el cine amplía los descuentos a mayores de 60 años así:

Menores de edad	6€
Mayores de 60 años	5€
Resto de mayores de edad	4€

En ese caso, las asignaciones condicionales se pueden anidar de forma bastante sencilla colocando en columna las condiciones y sus correspondientes valores.

```
import java.util.*;
public class Ejemplo19{
    public static void main(String[] args){
        System.out.println("Introduzca la edad");
        int edad = new Scanner(System.in).nextInt();
        int descuento = edad < 18 ? 6 : // si edad < 18, guardamos 6 en la variable
                                edad > 60 ? 5 : 4; // si edad > 60, guardamos 5 y si no, un 4
    }
}
```

Ejercicio 40: Haz un programa que pregunte al usuario por su número de DNI. El programa mostrará la letra que corresponde a dicho DNI, teniendo en cuenta que se obtiene así:

- Calculamos el resto de dividir el número de DNI entre 23
- La letra del DNI es la letra que corresponde a ese resto según la siguiente tabla

0 → T	1 → R	2 → W	3 → A	4 → G	5 → M
6 → Y	7 → F	8 → P	9 → D	10 → X	11 → B
12 → N	13 → J	14 → Z	15 → S	16 → Q	17 → V
18 → H	19 → L	20 → C	21 → K	22 → E	23 → I

8.- Ejercicios finales

Ejercicio 41: La Guardia Civil necesita un programa para multar y quitar puntos del carnet a los conductores que realizan una infracción.

- El programa comenzará preguntando:
 - Si lleva el carnet de conducir
 - La cantidad de puntos que tiene el usuario
 - El guardia hará un control y preguntará la tasa de alcoholemia en mg/l
- A continuación, el programa preguntará cuál de estas infracciones ha cometido

Infracción	Puntos	Multa
1 - Usar el móvil sujetándolo con la mano mientras se conduce	6	200€
2 - No llevar puesto el cinturón	4	200€
3 - Saltarse un semáforo en rojo	4	200€
4 - Circular en sentido contrario	6	500€
5 - Conducción temeraria	6	500€
6 - No respetar las señales de los agentes	4	200€

- El programa calculará la multa y los puntos que se retiran, de acuerdo a la infracción cometida. También se tendrán en cuenta estas penalizaciones, que podrán aplicarse con independencia de la infracción cometida

Infracción	Puntos	Multa
Si se conduce sin el carnet	0	100€
Si la tasa de alcoholemia está entre 0.25 y 0.5 mg/l	4	500€
Si la tasa de alcoholemia supera los 0.5 mg/l	6	1000€

- El programa finalizará mostrando los puntos retirados y la multa

Ejercicio 42: Realiza un programa en el que haya una variable entera llamada año inicializada con cualquier valor positivo. El programa mostrará por pantalla si el año guardado en dicha variable es bisiesto. Ten en cuenta que un año es bisiesto cuando se cumple alguna de estas dos condiciones:

- a) Es múltiplo de 400
- b) No es múltiplo de 400, pero es múltiplo de 4 y no es múltiplo de 100

Ejercicio 43: Hacer un programa en el que haya una variable entera llamada dinero e inicializarla a cualquier valor. El programa nos expresará esa cantidad en billetes de 500, 100, 50, 20 y 10 € y monedas de 2 y 1 €. Se ignoran los céntimos.

Sugerencia: Antes de programar, hacer un ejemplo con lápiz y papel. Una vez que tengas claro el procedimiento que hay que seguir para resolver el problema planteado, traslada a un programa las mismas operaciones que realizas sobre el papel.

Ejercicio 44: Estamos haciendo un programa de loterías y tenemos dos variables llamadas número y NúmeroPremiado, que se encuentran inicializadas con números de 5 cifras. El programa deberá mostrar por pantalla:

- a) Si el número coincide con el número premiado.
- b) Si el número tiene reintegro, es decir, se dan a la vez las dos siguientes condiciones:
 - a. La primera cifra del número coincide con la primera cifra del número premiado
 - b. La última cifra del número coincide con la última cifra del número premiado

Sugerencia: Obtén el cociente y resto de dividir el número premiado entre 10, y mira si tienen algo que ver con su última cifra. Inventa algo parecido para obtener la primera cifra.

Ejercicio 45: Un equipo de 7 amigos futbolistas llamados Pedro, Luis, Miguel, Jorge, Juan, Manolo y Pepe echa a suerte quien se pone de portero. Para eso, todos sacan a la vez la mano y se cuenta el número de dedos que ha sacado cada uno. Entonces, se va contando de uno en uno y al que le toque, se pone de portero. Si al llegar al último jugador no se ha terminado, se sigue contando comenzando por el primero.

Realiza un programa que muestre por pantalla el nombre del jugador que se pone de portero si el número de dedos que ha sacado cada amigo es:

4	5	1	0	3	2	3
---	---	---	---	---	---	---